

SIRE- Final Report

PROG 3300 – Integrated Project for Programming IT Programming – Year 2

(SIRE)

Simulated Incident Response Environment

A Real-Time Training Platform for Emergency Scenarios

Prepared By:

~ Nibble ~

Tom Burchell

Leon Wasiliew

John Hay

Kael Payette

Project Sponsor:

Alfred Parks

Institution:

Nova Scotia Community College

IT Programming Program

Table of Contents

1. Introduction.....	7
1.1. Purpose of the Report.....	7
1.2. Project Overview	7
1.3. Significance of the Project.....	7
2. Project Charter.....	8
2.1. Project Description.....	8
2.2. Objectives and Goals.....	8
2.3. Problem Statement.....	8
2.4. Scope Statement	9
2.4.1. In Scope	9
2.4.2. Out of Scope	9
2.5. Constraints.....	9
2.6. Assumptions	10
2.7. Risks	10
2.7.1. General Project Risks.....	10
2.7.2. Technical and External Risks	10
2.8. Deliverables.....	10
2.9. Early Requirements Analysis	11
2.9.1. Functional Requirements.....	11
2.9.1. Non-Functional Requirements	11
2.9.3. User Requirements	11
3. Team Charter	12
3.1. Team Name	12
3.1.1. Winner	12
3.1.2. Candidates.....	12
3.2. Team Members and Roles	12
3.2.1. Tom Burchell – Backend Developer	12
3.2.2. Leon Wasiliew – Frontend Developer	12
3.2.3. John Hay – Script Master	12

3.2.4. Kael Payette – Project Manager	12
3.3. Team Values.....	13
3.4. Working Agreements.....	13
3.4.1. Meetings.....	13
3.4.2. Decisions.....	13
3.4.3. Participation and Communication	13
3.4.4. Code and Documentation Standards.....	13
3.5. Conflict Resolution	14
3.6. Communication Tools.....	14
3.7. Decision-Making Process	14
3.7.1. Project Scope Decisions	14
3.7.2. Role Scope Decisions.....	14
4. Software Requirements Specification	15
4.1. Functional Requirements.....	15
4.2. Non-Functional Requirements.....	15
4.3. User Requirements	16
4.4. Interface Requirements	16
4.4.1. User Interface Requirements.....	16
4.4.2. Software Interfaces	16
4.4.3. Communication Interfaces	16
4.5. Data Requirements.....	17
4.5.1. Data Types.....	17
4.5.2. Data Flow.....	17
4.6. Use Case	17
5. System Design & Architecture	18
5.1. System Architecture Overview	18
5.2. Component Design	18
5.3. Data Design.....	18
5.4. UI/UX Design.....	18
5.5. Diagrams	19

5.5.1. Use Case Diagram.....	19
5.5.2. Entity Relationship Diagram.....	20
5.5.3. Component Diagram.....	21
5.5.4. Activity Diagram (Admin).....	22
5.5.5. Activity Diagram (Trainee)	24
5.5.6. Home Page Wireframe Diagram	26
5.5.7. Home Page Mockup.....	27
6. System Implementation.....	28
6.1. Coding Standards and Best Practices	28
6.2. Version Control and Collaboration	30
6.2.1. Tools Used and Rationale	30
6.2.2. Branching Strategy.....	30
6.2.3. Collaboration Workflow	32
6.3. Key Code Implementations.....	32
6.3.1. Authentication System.....	32
6.3.2. API Communication and Data Handling	32
6.3.3. Session Management System.....	33
6.3.4. Scenario Decision System	33
6.3.5. Reusable Components and Layout Architecture	33
6.3.6. Backend Logging and Security.....	33
6.3.7. Third-Party Libraries	34
6.4. Database Implementation.....	34
6.4.1. SQL Database for User Management.....	34
6.4.2. JSON for Scenario and Application Data	37
6.5. User Interface (UI) Design	39
6.5.1. Key UI Components.....	39
6.5.2. Reusable Components	40
6.5.3. User Interaction and Navigation.....	40
6.5.4. Design Approach	40
6.5.5. Visual Design.....	40

7. Testing	43
7.1. Testing Types	43
7.1.1. Frontend	43
7.1.2. Backend	43
7.2. Test Cases and Results	43
7.2.1. Frontend	43
7.2.2. Backend	44
8. Conclusion	57
8.1. Overview	57
8.2. What Went Well	57
8.3. What Went Wrong	57
8.4. What We Would Do Differently	57
9. References	58
Appendix A – Gantt Chart	60
Table Overview	60
Diagram Overview	63
Appendix B – GitHub Repository	64
Repository Structure	64
Commit Standards	64
Appendix C – Project Minutes	65
Weeks 1-4	65
Weeks 5-8	65
Weeks 9-13	65
Appendix D – Project Meetings	66
Meeting #1	66
Project Details	66
Agenda Items	66
Project Status Updates	67
Risks/Issues Identified	68
Next Meeting	68

Meeting #2	69
Project Details	69
Agenda Items	69
Project Status Updates	70
Risks/Issues Identified	70
Next Meeting	70
Meeting #3	71
Project Details	71
Agenda Items	71
Project Status Updates	72
Risks/Issues Identified	72
Next Meeting	72

1. Introduction

1.1. Purpose of the Report

The purpose of this final report is to provide a comprehensive overview of the development of the **Simulated Incident Response Environment (SIRE)** project. This report consolidates all phases of the Software Development Life Cycle (SDLC), including planning, design, implementation, testing, and deployment. It serves as a formal documentation artifact that demonstrates the technical, collaborative, and problem-solving skills developed throughout the project lifecycle.

1.2. Project Overview

The Simulated Incident Response Environment (SIRE) is a **full-stack web-based application** designed to simulate emergency scenarios in real time. The system allows users to engage in decision-making exercises under simulated pressure, replicating real-world emergency response conditions.

The platform consists of two primary roles:

- **Administrator:** Controls and monitors simulation sessions
- **Trainee:** Participates in scenarios and makes decisions

The application integrates real-time communication technologies to ensure synchronized experiences across all users.

1.3. Significance of the Project

Traditional emergency preparedness training methods such as drills and tabletop exercises present several challenges:

- High cost
- Safety risks
- Limited accessibility
- Lack of realism

SIRE addresses these issues by providing:

- **A cost-free solution**
- **A safe and controlled training environment**
- **A platform-independent web application**
- **A dynamic workflow with real-time interaction**

This approach enhances learning outcomes by allowing users to practice decision-making without real-world consequences.

2. Project Charter

2.1. Project Description

Emergency preparedness training is essential for workplaces, schools, and organizations.

The Simulated Incident Response Environment (SIRE) is a digital training application designed to deliver realistic, scenario-based emergency simulations. By leveraging real-time communication technology, interactive decision-making, and dynamic escalation, the simulator provides a safe, cost-efficient, high-fidelity training environment for practicing incident response.

The system supports both administrators, who guide scenarios, and trainees, who make decisions under time pressure.

2.2. Objectives and Goals

The primary objective of the Simulated Incident Response Environment (SIRE) project is to create a scalable, modular incident response training tool that can grow in complexity as development progresses. The project is intentionally designed with expandable scenario modules, allowing the team to meet the required scope while still leaving room for enhancements.

The goal for this project is to deliver a functional prototype that demonstrates the core features defined in the project scope. The system does not need to be fully polished, but it must support the essential workflow of creating a session, delivering injects, enabling trainee decision-making, and providing feedback.

Success is measured by the ability to deliver these core components in a usable prototype that accurately represents the intended training experience.

2.3. Problem Statement

Traditional emergency preparedness methods, such as full-scale physical drills, are costly, disruptive, and sometimes unsafe. Tabletop exercises often lack realism and meaningful engagement. Many institutions conduct drills infrequently due to scheduling challenges, liability concerns, limited resources, or a lack of trained instructors. As a result, individuals and teams are typically underprepared for high-risk incidents such as fires, active threats, power outages, structural failures, medical emergencies, and hazardous material events.

There is a clear need for a scalable, accessible, and high-fidelity training solution that overcomes these limitations and better prepares trainees for these emergencies.

2.4. Scope Statement

This project involves designing, developing, testing, and hosting the Simulated Incident Response Environment (SIRE), a full-stack web application created by team Nibble: Tom Burchell, Leon Wasiliew, John Hay, and Kael Payette. The application will provide administrators with tools to configure and run escalating incident scenarios, while trainees will use a dedicated interface to practice responding to real-time incidents.

The project will follow an Agile (SCRUM) approach over a 15-week schedule.

2.4.1. In Scope

- Development of a full-stack JavaScript web application
- Design of the backend and frontend architecture
- Use of Node.js, Express, Socket.IO, and React
- Implementation of an Administrator Dashboard
- Implementation of a Trainee Interface
- Development of four fully functional emergency scenarios (out of eight)
- Real-time event logging and synchronization
- Session-based access for administrators and trainees
- Hosting the application on a free-tier cloud provider
- Creation of required project documentation and artifacts
- Additional features approved by the Sponsor and PM during sprint planning

2.4.2. Out of Scope

- Enterprise-level scalability (beyond small-group training of ~10 trainees)
- Paid hosting, premium cloud compute instances, or long-term server maintenance
- Cybersecurity hardening beyond basic best practices
- Mobile apps (iOS / Android), as this version is limited to the web
- Scenario packs beyond the initial four (future stretch goals only)
- Post deployment feature updates after the academic term concludes
- Features not approved by the Sponsor and PM during sprint planning

2.5. Constraints

- **Time Constraint:** A fixed 15-week academic schedule that limits extended testing.
- **Resource Constraint:** A team of four students with limited full-time availability.
- **Technical Constraint:** The application must operate within free-tier cloud hosting and other free platforms.
- **Knowledge Constraint:** Students are learning full-stack JavaScript and real-time communication while building the application.
- **Risk Constraint:** Real-time synchronization introduces complexity and potential latency challenges.

2.6. Assumptions

We assume the following to be true:

- Free-tier hosting services such as Vercel, Railway, and Render remain available and stable throughout the duration of the project.
- All team members are consistently available for development tasks.
- JavaScript-based full-stack environment is stable and compatible with project needs.
- Academic requirements and scope remain stable without major changes.
- Team commits regularly to GitHub and follows standard workflow practices.

2.7. Risks

2.7.1. General Project Risks

- **Time Limitations:** The fixed academic schedule may restrict testing and scenario development.
- **Scope Creep:** Additional feature ideas may emerge during development and exceed the initial project scope.

2.7.2. Technical and External Risks

- **Service Provider Instability:** Free-tier hosting platforms may experience policy changes that affect deployment.
- **Real-Time Communication Issues:** WebSocket-based synchronization may introduce latency or inconsistent state between trainees.
- **Limited Experience:** The learning curve regarding creating a full-stack JavaScript application may slow development or require additional time for troubleshooting.

2.8. Deliverables

- **Functional real-time incident simulator** (frontend + backend + WebSocket engine)
- **Four complete emergency-response scenario scripts** with escalation logic
- **Documentation package**, including:
 - README
 - User guides
 - Technical architecture
 - Deployment instructions
- **Presentation materials** and **demonstration video**
- **Fully deployed system** accessible via public URLs
- **Testing plan and results**
- **Project management artifacts** (Gantt chart, meeting minutes, interim reports)

2.9. Early Requirements Analysis

2.9.1. Functional Requirements

The system must support:

- Real-time scenario updates
- User session management
- Decision tracking and branching logic
- Admin controls for simulation flow

Example:

- The system shall allow administrators to start, pause, and end scenarios
- The system shall broadcast events to all trainees in real time

2.9.1. Non-Functional Requirements

- Low-latency communication
- High usability
- Maintainable code structure
- Secure authentication practices

2.9.3. User Requirements

Administrator:

- Create sessions
- Monitor participants
- Control simulation flow

Trainee:

- Join sessions
- Receive events
- Make decisions
- Receive feedback

3. Team Charter

3.1. Team Name

3.1.1. Winner

- Nibble

3.1.2. Candidates

- The Floor Horseman
- Backend Bois
- Quarter Byte

3.2. Team Members and Roles

The team selected roles based on individual strengths and observed performance in previous projects. The Project Manager role was chosen through team consensus.

3.2.1. Tom Burchell – Backend Developer

Tom is responsible for designing and maintaining the backend architecture. His work includes implementing real-time communication using Socket.IO and developing the scenario escalation logic that drives the simulation.

3.2.2. Leon Wasiliew – Frontend Developer

Leon is responsible for building the React-based frontend and shaping the user interface layouts. He ensures the application remains accessible for all user roles and maintains reliable communication between the server and clients.

3.2.3. John Hay – Script Master

John is responsible for researching and writing the scenario scripts that define the training experience. He also conducts integration testing and ensures the system meets both usability and accessibility standards.

3.2.4. Kael Payette – Project Manager

Kael oversees team coordination and ensures smooth progression. He manages documentation standards, facilitates communication, keeps track of minutes, and oversees Git version control workflows to maintain consistency across the project.

3.3. Team Values

These values guide how the team collaborates and approaches the project:

- **Respect** – Support the ideas, time, and working styles of all team members.
- **Accountability** – Take ownership of responsibilities and follow through on commitments, including communicating delays or challenges.
- **Clarity** – Communicate openly and ensure all team members understand their roles and expectations of their contributions.
- **Quality** – Deliver work that aligns with project standards and requires minimal revision.
- **Adaptability** – Adjust plans when needed and remain flexible within the Agile approach.

3.4. Working Agreements

These agreements define how the team will operate throughout the project:

3.4.1. Meetings

The team will meet twice weekly (each 10-15 minutes) in person at the college to review previous, current, and next action items. These meetings will default to the scheduled PROG3300 class days.

- Team members must notify the group if they are unable to attend.
- Absences will be recorded in the Timesheet document.
- If a meeting is cancelled due to holidays, weather, or campus closures, it will be moved to the next available school day.

3.4.2. Decisions

All major decisions will be discussed as a team, with the Project Manager facilitating the process.

- Major decisions include, but are not limited to, changes to the project scope, significant document revisions, and adjustments to team roles.

3.4.3. Participation and Communication

Team members will actively engage in the project and provide regular progress updates through the agreed-upon communication channels.

- Extended inactivity without communication will be addressed by the team.

3.4.4. Code and Documentation Standards

All code and documentation contributions will follow established conventions.

- Contributions that do not meet the agreed standards must be revised by the contributor.

3.5. Conflict Resolution

1. Conflicts will first be addressed through direct, respectful discussion between the team members involved.
2. If the issue remains unresolved, it will be escalated to the Project Manager for mediation.
 - If the Project Manager is directly involved in the conflict, the matter will be escalated to the Project Sponsor/Project Overseer, Alfred Parks.
3. All resolutions will prioritize fairness and maintaining a cohesive team environment.

3.6. Communication Tools

This team will use the following tools to support collaboration and project coordination:

- **GitHub** – Version control for the source code and documentation.
- **Jira** – Sprint planning, task management, and reporting.
- **Microsoft Teams** – Day-to-day communication among team members.
- **Outlook/Gmail** – Formal communication with the Project Sponsor and external stakeholders.

3.7. Decision-Making Process

Decision-making depends on the domain of the decision:

3.7.1. Project Scope Decisions

These decisions involve changes that affect the direction and scope of the project.

- Decisions will be made collaboratively through team voting.
- The Project Manager will facilitate the process.
- The Project Sponsor may be consulted about major changes or clarifications.

3.7.2. Role Scope Decisions

These decisions relate to individual responsibilities and workflow preferences.

- Decisions are made by the individual responsible for the role or task.
- Team members are expected to communicate impacts or concerns.
- The team may provide input if the decision affects shared work or dependencies.

If the outcome of a role scope decision will potentially affect many smaller operations, it may be escalated and treated as a project scope decision.

4. Software Requirements Specification

4.1. Functional Requirements

These requirements define the core capabilities the application must deliver.

- **FR1:** The application shall load predefined emergency scenarios.
- **FR2:** The application shall display the scenario overview and the defaulted time to complete.
- **FR3:** The application shall allow administrators to start, pause, resume, and end scenarios.
- **FR4:** The application shall trigger injects at scheduled times.
- **FR5:** The application shall broadcast events to all participating trainees in real-time.
- **FR6:** The application shall present decision options to trainees.
- **FR7:** The application shall record trainee decisions and update the scenario state.
- **FR8:** The application shall trigger branching events based on scenario logic rules.
- **FR9:** The application shall synchronize scenario state across all connected clients.
- **FR10:** The application shall handle user join and leave events during an active session.
- **FR11:** The application shall allow administrators to monitor trainee decisions.
- **FR12:** The application shall allow administrators to trigger escalations manually.
- **FR13:** The application shall display feedback based on decisions.
- **FR14:** The application shall generate dynamic session keys for access.
- **FR15:** The application shall track user sessions for the duration of the scenario.

4.2. Non-Functional Requirements

These requirements define the qualitative attributes and performance benchmarks:

- **NFR1:** The system shall deliver real-time updates with acceptable latency.
- **NFR2:** The system shall support a maximum of 10 concurrent trainees.
- **NFR3:** The system shall maintain scenario state until the session has ended.
- **NFR4:** The system shall prevent loss of event data during normal operation.
- **NFR5:** The system shall provide accessible interfaces for both administrators and trainees.
- **NFR6:** The system shall require minimal training of users to operate effectively.
- **NFR7:** The system shall not store sensitive data.
- **NFR8:** The system shall enforce basic access control for administrator-level functions.
- **NFR9:** The system shall use clear text and readable contrast to support usability.
- **NFR10:** The system shall have a codebase structured for demonstration and handoff.
- **NFR11:** The system shall include clear documentation and follow a modular design.

4.3. User Requirements

These requirements define what users need to accomplish within the system.

- **UR1:** The administrator shall be able to create and manage a session.
- **UR2:** The administrator shall be able to configure the scenario settings.
- **UR3:** The administrator shall be able to control the scenario flow.
- **UR4:** The administrator shall be able to trigger escalations during a scenario.
- **UR5:** The trainee shall be able to join a session using a generated key.
- **UR6:** The trainee shall be able to receive events in real time.
- **UR7:** The trainee shall be able to make decisions in response to events.
- **UR8:** The trainee shall be able to receive feedback based on their decisions.

4.4. Interface Requirements

4.4.1. User Interface Requirements

Administrator Interface

- Provides a dedicated view built for overseeing and guiding the scenario.
- Shows real-time updates for active events and scenario progress.
- Offers clear navigation for starting, pausing, escalating, and monitoring the scenario.

Trainee Interface

- Provides a view centered on receiving events and responding to them.
- Shows real-time alerts for new injects and decisions that require attention.
- Uses simple navigation and a clean layout that supports quick responses.

4.4.2. Software Interfaces

- Socket.IO for real-time communication
- Node.js backend
- React frontend

4.4.3. Communication Interfaces

- WebSocket for real-time updates
- HTTPS for secure communication

4.5. Data Requirements

4.5.1. Data Types

- Scenario and inject data shall be stored in JSON (JavaScript Object Notation) or a similar format suitable for a JavaScript-based application.
- No personal or sensitive information about the trainee shall be stored beyond basic identifiers (e.g., name, initials).
- System state shall persist long enough to support reconnections during an active scenario.
- Data shall be accessible to both the administrator and trainees in real time.

4.5.2. Data Flow

- Injects shall flow from the server to all connected clients simultaneously.
- Trainee decisions shall be sent to the server and reflected in the administrator dashboard.
- Scenario state updates shall be broadcast to all users without delay.
- Administrator actions (e.g., start, pause, escalate) shall update the system state immediately.

4.6. Use Case

Use Case	Actor	Precondition	Includes/Extends
UC1: Create Session	Administrator	Email is entered	-
UC2: Start Scenario	Administrator	Scenario is selected	Includes – UC3
UC3: Load Scenario Data	Administrator	Scenario files can be accessed	-
UC4: Trigger Escalation	Administrator	Scenario is running	Extends – UC2
UC5: Join Session	Trainee	Valid session is online	Extends – UC1
UC6: Receive Inject	Trainee	Scenario is running	-
UC7: Submit Decision	Trainee	Inject requires a decision	Includes: UC6
UC8: View Feedback	Trainee	Decision has been entered	Extends – UC7
UC9: Edit Scenario Files	Developer	Developer has access to files	-
UC10: Validate Structure	Developer	Scenario file has been edited	Includes – UC9
UC11: Deploy Updates	Developer	Scenario validation is accurate	Extends – UC10

5. System Design & Architecture

5.1. System Architecture Overview

The system follows a **client-server architecture**:

- **Frontend**: React (UI & interaction)
- **Backend**: Node.js + Express (logic & API)
- **Real-Time Layer**: WebSockets (Socket.IO)

This architecture ensures scalability and separation of concerns.

5.2. Component Design

The system is divided into:

- Authentication Module
- Session Management Module
- Scenario Engine
- UI Layer

Each component operates independently, improving maintainability.

5.3. Data Design

SQL Database:

- Stores user credentials
- Ensures data integrity

JSON Data:

- Stores scenario logic
- Enables flexible content updates

5.4. UI/UX Design

The user interface follows a **component-based design approach**:

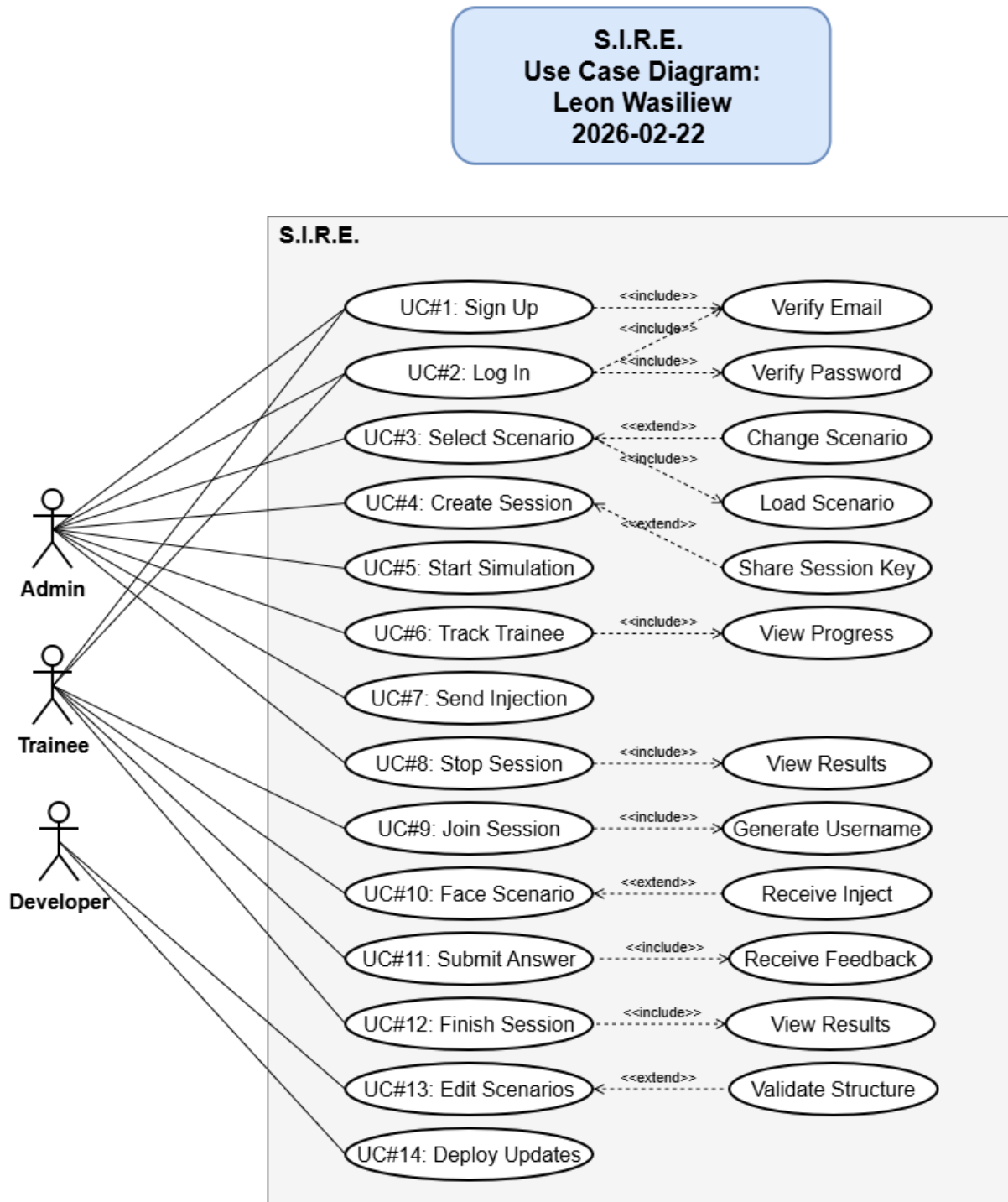
- Reusable UI components
- Clear visual hierarchy
- Role-based layouts

Design principles include:

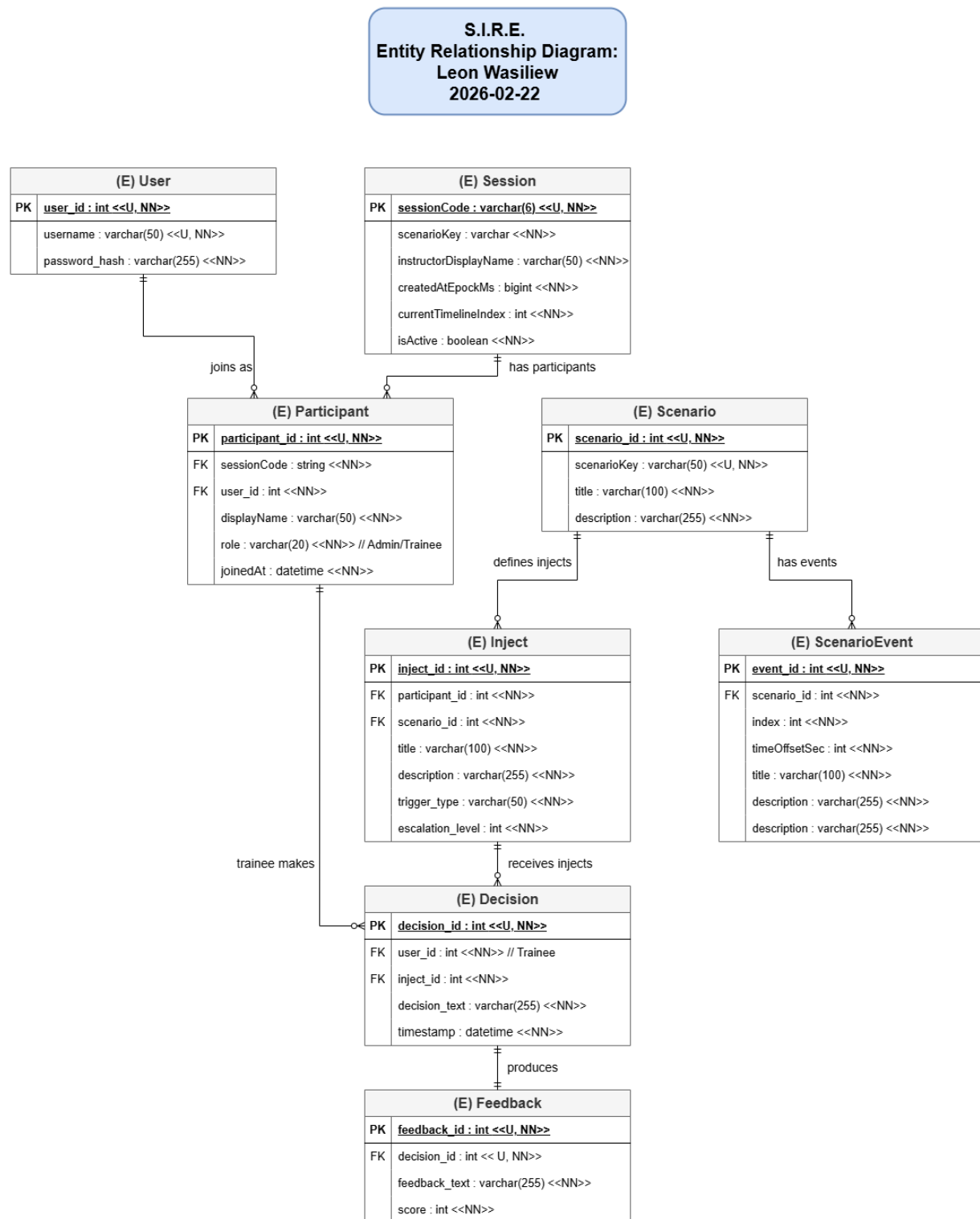
- Simplicity
- Accessibility
- Immediate feedback

5.5. Diagrams

5.5.1. Use Case Diagram

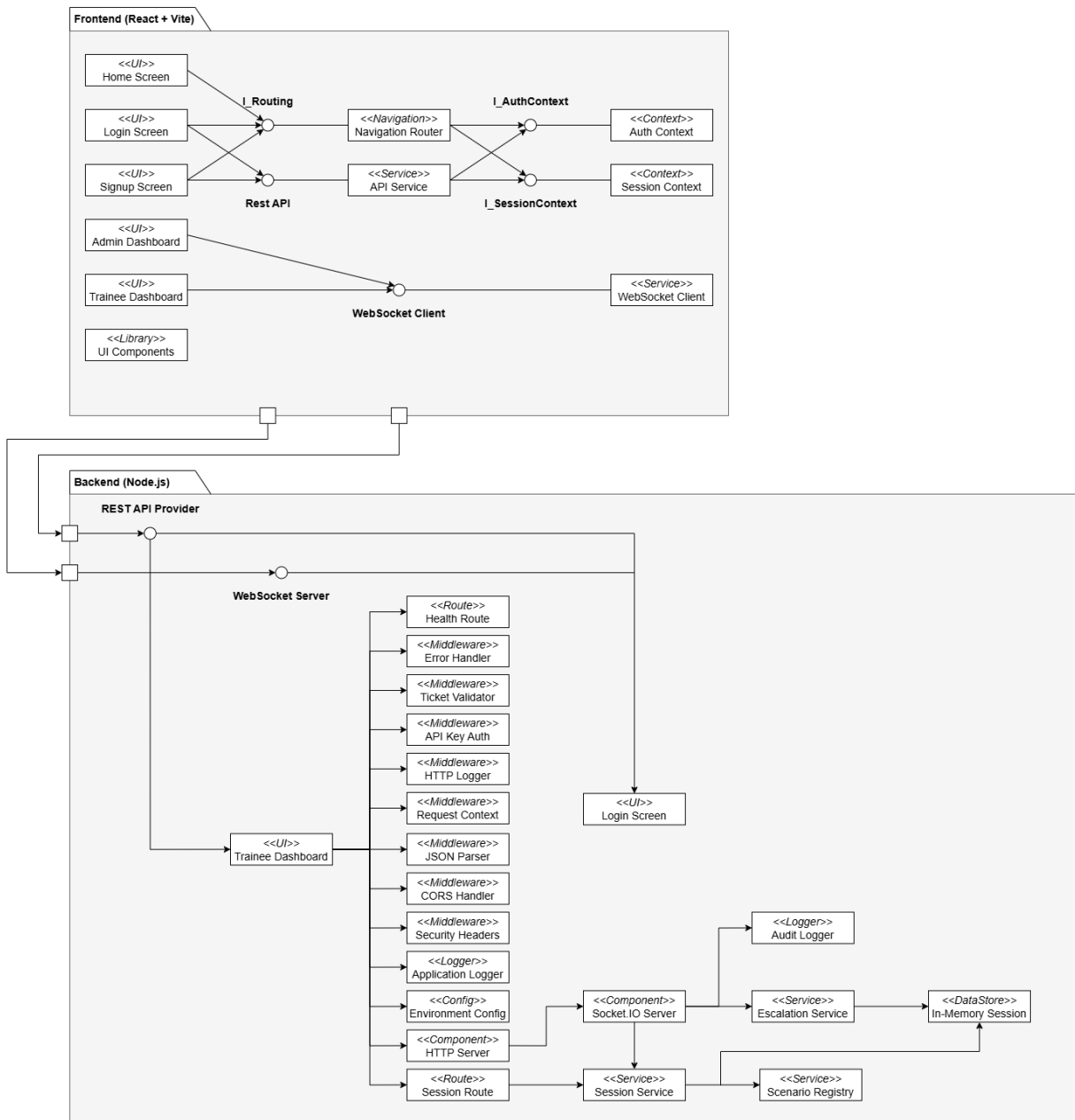


5.5.2. Entity Relationship Diagram



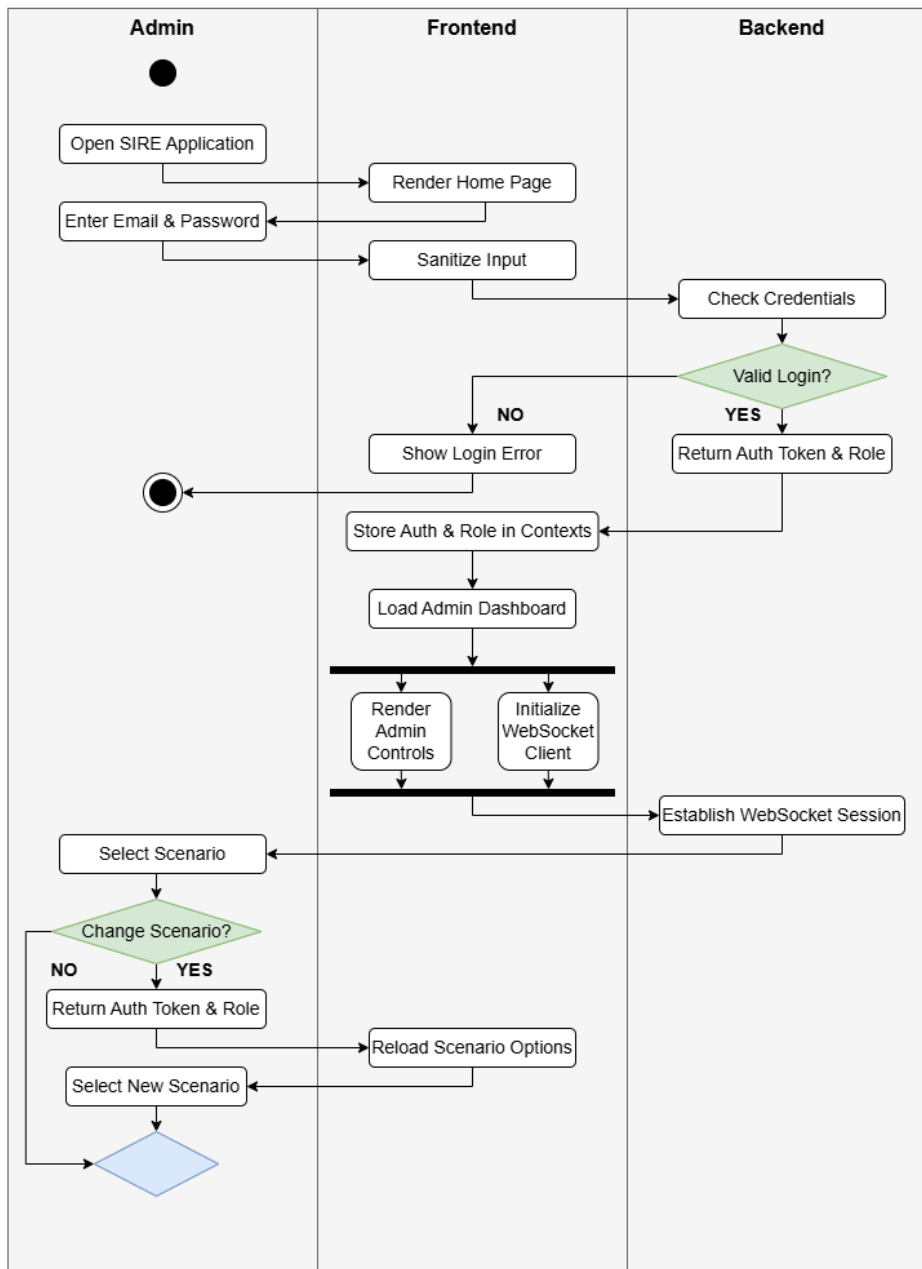
5.5.3. Component Diagram

S.I.R.E.
Component Diagram:
Leon Wasiliew
2026-02-22

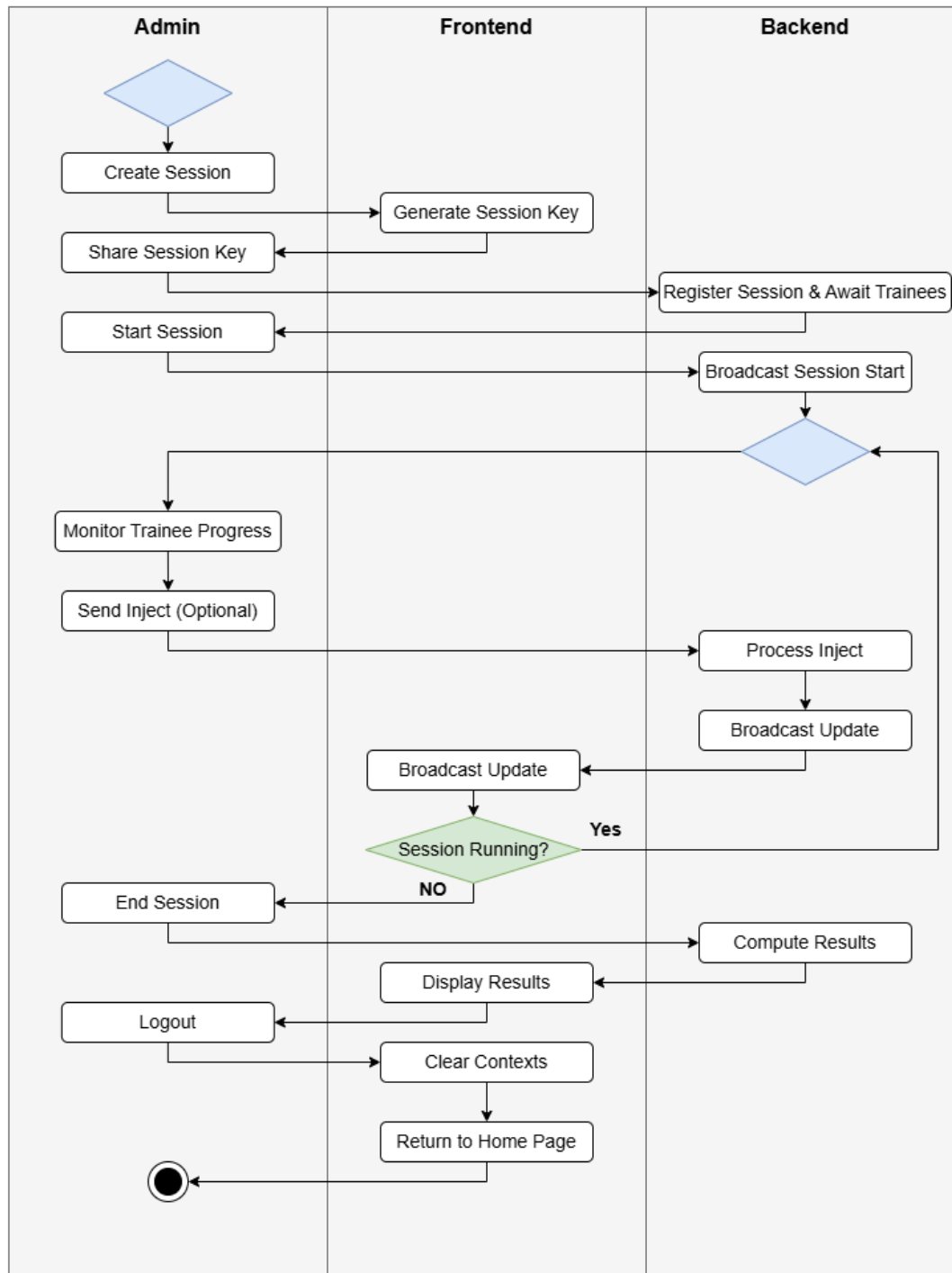


5.5.4. Activity Diagram (Admin)

S.I.R.E.
Activity Diagram (Admin #1):
Leon Wasiliew
2026-02-22

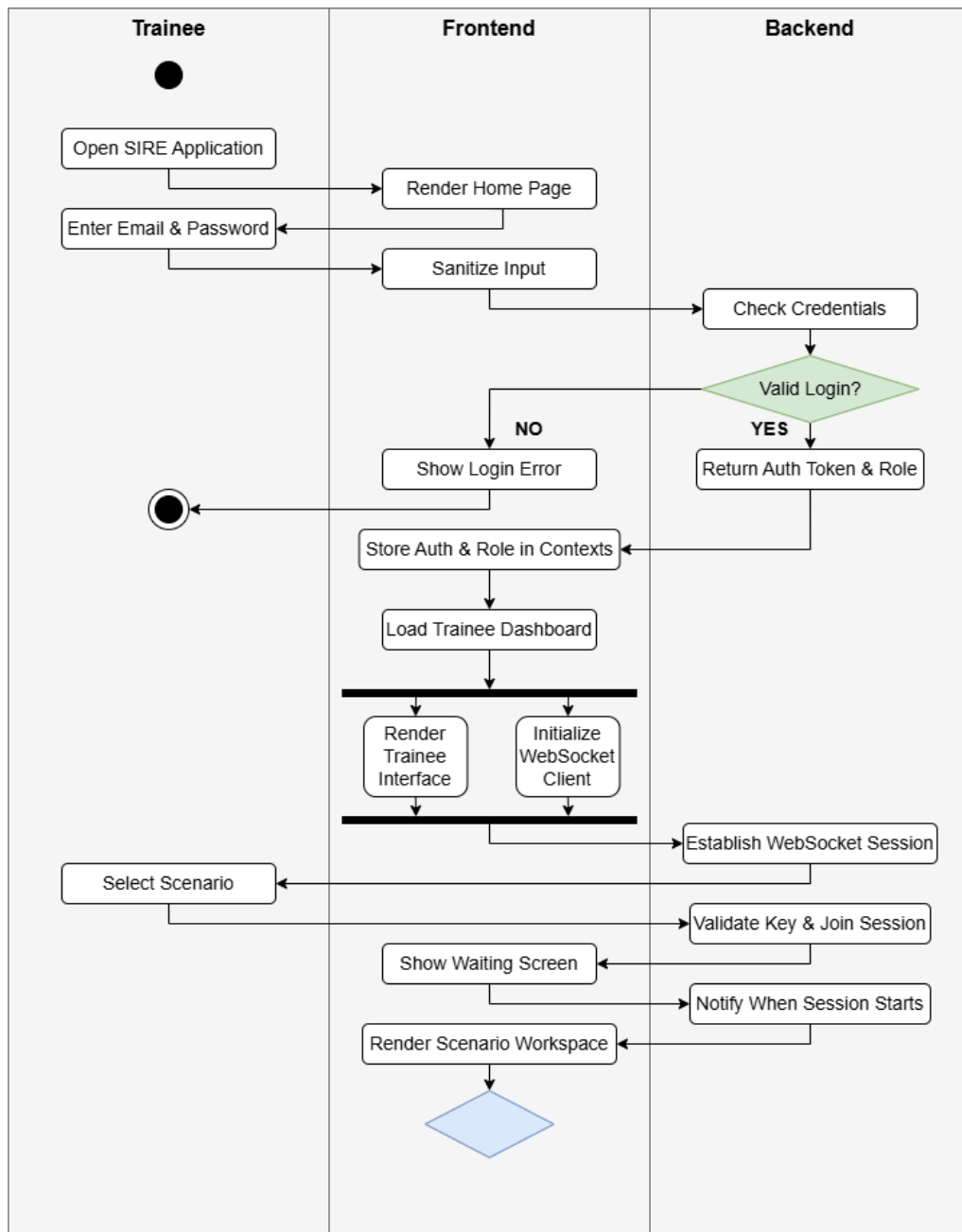


S.I.R.E.
Activity Diagram (Admin #2):
Leon Wasiliew
2026-02-22

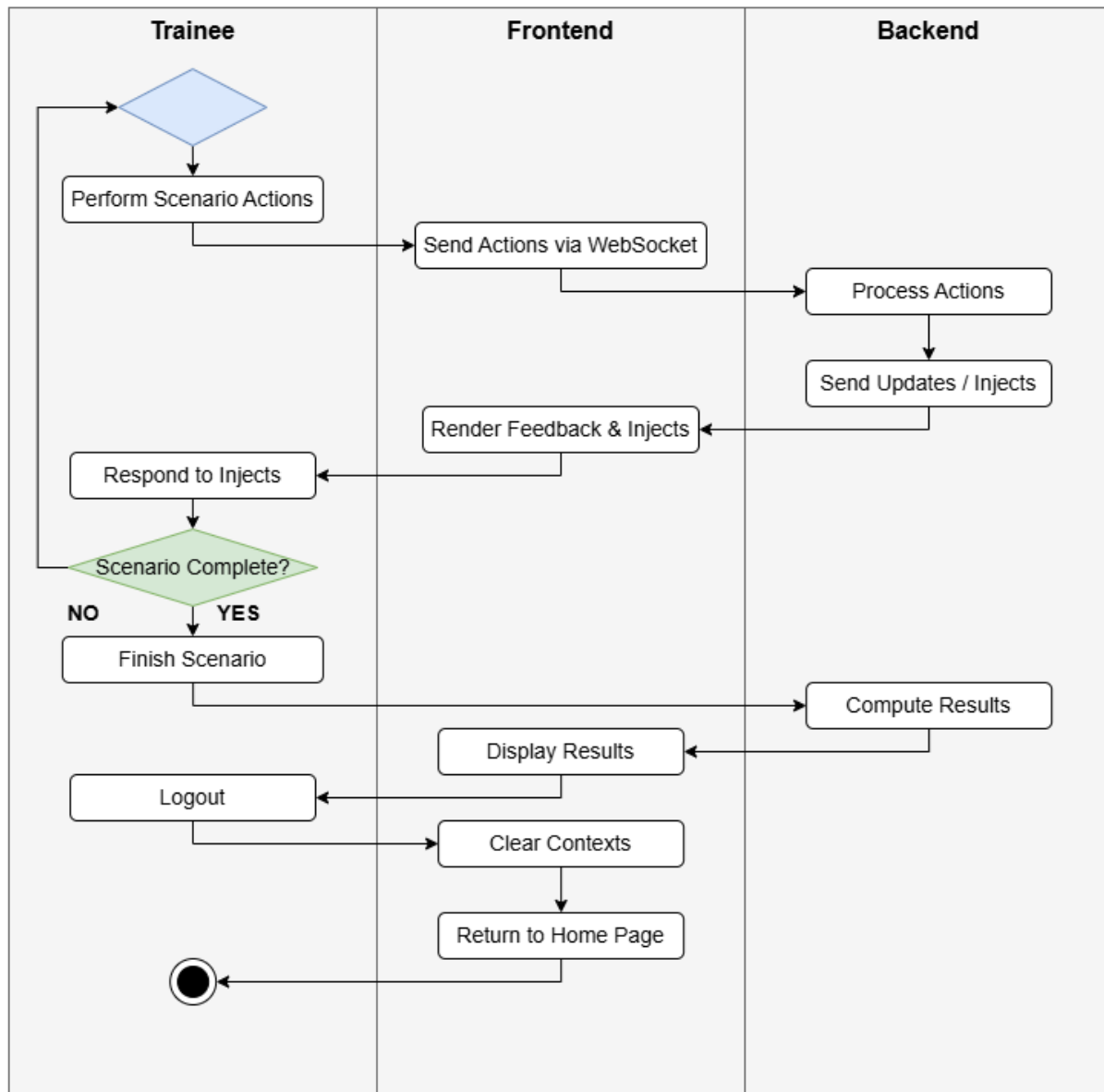


5.5.5. Activity Diagram (Trainee)

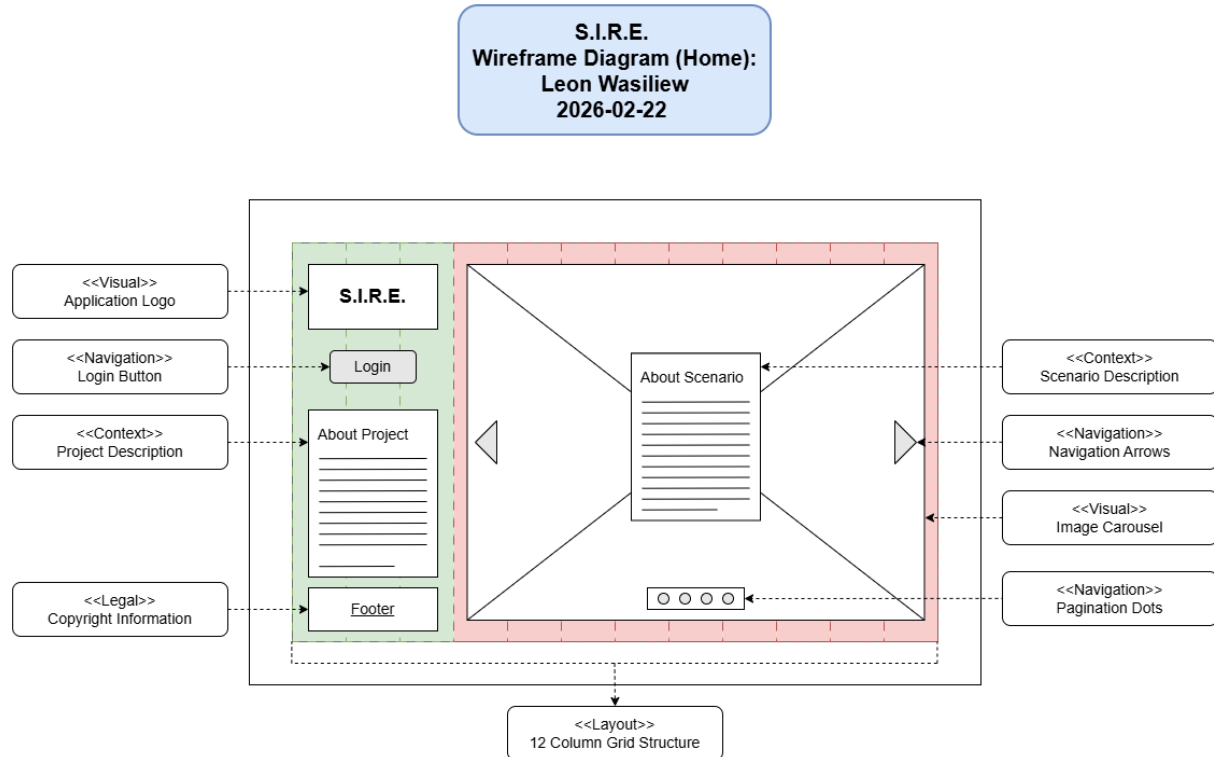
S.I.R.E.
Activity Diagram (Trainee #1):
Leon Wasiliew
2026-02-22



S.I.R.E.
Activity Diagram (Trainee #2):
Leon Wasiliew
2026-02-22



5.5.6. Home Page Wireframe Diagram



5.5.7. Home Page Mockup

**S.I.R.E.
Mockup (Home):
Leon Wasiliew
2026-02-22**



6. System Implementation

6.1. Coding Standards and Best Practices

Category	Description	Example
Coding Standards	Rules that ensure code is written consistently across the entire project, including formatting, spacing, and syntax style.	Frontend & Backend: Consistent indentation formatting, and clear separation of JSX and logic in React components, as well as consistent function and object formatting in backend controllers.
Naming Conventions	Guidelines for naming variables, functions, components, and files in a clear and meaningful way.	Frontend: React components use PascalCase (e.g., HomeLayout), non-default functions and variables use camelCase (e.g., setScenarios), and CSS classes use kebab-case (e.g., home-container). Backend: Functions and variables follow camelCase naming (e.g., sessionController).
Project Structure	Organizing files into logical folders to separate concerns and improve maintainability.	Frontend: Organized into folders under src, including assets, components, context, hooks, layouts, navigation, screens, services, and utils. Backend: Organized into folders under src, including config, controllers, middleware, models, routes, scenarios, services, sockets, and utils.
Clean Code	Writing simple, readable code where each function or module has a single responsibility.	Backend: Validation logic is separated into utility functions instead of being embedded directly in controllers, keeping controller methods clean and focused.
WET Principle	Write Everything Twice (WET) Principle allows similar code to exist in multiple places when small differences make reuse impractical.	Frontend: Multiple layout or UI sections are written separately (e.g., CreateSession & JoinSession for sessions) to allow unique content and structure for each scenario.

DRY Principle	Don't Repeat Yourself (DRY) Principle avoids repetition by reusing logic or components across the application.	Frontend: Reusable components (e.g., buttons or layout elements) are used across multiple screens. Backend: Shared validation and normal normalization functions are reused across different controller methods.
Error Handling	Managing errors to prevent crashes and provide meaningful feedback to users and developers.	Backend: Controller methods use structured error handling (e.g., try-catch statements and conditional checks) and return appropriate HTTP status codes and messages when errors occur.
Frontend Best Practices	Recommended techniques for building maintainable and scalable React applications.	Frontend: Components are modular and separated from assets and styles, with modern external libraries used for UI functionality.
Backend Best Practices	Structuring backend logic in a modular and scalable way using the separation of concerns.	Backend: Controllers handle request/response logic, while services manage business logic, and utilities handle validation and helper functions.
API Design	Designing backend endpoints in a consistent and RESTful manner.	Backend: Endpoints follow REST principles, using routes for creating, retrieving, and deleting resources.
Comments & Documentation	Providing clear explanations for files, functions, and complex logic.	Frontend & Backend: Use of file-level, multi-line comments for description, along with function-level documentation and selective inline comments for complex logic.
Security & Validation	Ensuring inputs are validated and the application is protected from invalid or unsafe data.	Frontend: Form inputs include built-in HTML validation (e.g., required fields) to reduce invalid submissions. Backend: Request data is validated and normalized before processing, with invalid inputs rejected using appropriate responses.

6.2. Version Control and Collaboration

6.2.1. Tools Used and Rationale

Git & GitHub

GitHub served as our central platform for version control, documentation storage, and collaborative development. The repository “sire-simulator” was created at the start of the project, including a “.gitignore”, a proper folder structure, and access permissions for all team members. This ensured consistent organization and prevented untracked or unnecessary files from entering the codebase.

The repository contains folders for backend, frontend, documentation, scenarios, and testing. This supports modular development and provides a clear separation of responsibilities.

Microsoft Teams

Teams was used for our day-to-day communication, quick updates, and coordination between in-person meetings. The Teams group chat “The Floor Horseman” has served as our default communication channel. It has supported rapid clarification of tasks, sharing of progress, and alignment across all aspects of the development work.

Jira Software

Jira was used for sprint planning, task tracking, and workflow visibility. A full Jira project was created with epics for Project Management, Documentation, Testing, Deployment, Frontend, Backend, and Scenarios. Each team member was assigned to relevant epics, and the Jira board was linked to the GitHub repository to enable automatic issue tracking and commit association.

6.2.2. Branching Strategy

The team adopted a structured branching template to maintain code quality and prevent conflicts. Both the “main” and “dev” branches require pull request (PR) reviews. No direct pushes are allowed:

- **main:** The stable branch containing finalized, working code or the final versions of documentation. Only merges that are approved by other teammates are allowed.
- **dev:** The integration branch where all feature branches are merged before being promoted to main. This branch contains active development work.
- **feature branches:** Created per task using the format:
 - feature/<developer>/<feature-name>

Examples include:

- feature/tom/socket-engine
- feature/leon/admin-dashboard
- feature/john/scenario-fire
- feature/kael/documentation

Commit standards were also defined early in the project. Good commits required clear, descriptive messages and followed this structure: **(feat/fix/etc/)** + **(name)** + **(description)**.

Types include:

Type	Meaning	When to Use	Example Commit Message
feat	New Feature	Adding a new feature	feat: Add login page
fix	Bug Fix	Fixing a bug	fix: Resolve crash issue
docs	Maintenance tasks	Updating docs or README	docs: Update user guide
chore	Code style	General cleanup or upkeep	chore: Reorganize project files
style	Code style changes	Formatting-only changes	style: fix indentation
refactor	Code refactoring	Improving code without changing behavior	refactor: optimize auth logic
test	Adding or updating	Writing or modifying tests	test: Add unit tests
perf	Performance improvements	Enhancing speed or efficiency	perf: Improve load time.
build	Build System	Modifying build tools or dependencies	build: update dependencies

6.2.3. Collaboration Workflow

The team followed a consistent workflow combining GitHub, Jira, and Teams:

- Tasks were created and assigned in Jira, grouped under epics such as Backend Architecture, Frontend UI, and Scenario Development.
- Developers created feature branches for their assigned tasks and pushed updates regularly.
- Pull requests were used to merge work into the dev branch, ensuring review and integration before reaching main.
- Teams was used to coordinate progress, clarify responsibilities, and resolve blockers quickly.
- In-person meetings (twice weekly) were used to review progress, plan next steps, and ensure alignment across all roles.

6.3. Key Code Implementations

6.3.1. Authentication System

The full-stack JavaScript application employs a client-server approach for authentication, where the frontend communicates with backend API endpoints for login and signup. On the frontend, this is handled through `api.js`, `apiClient.js`, and the Login and Signup screens.

When a user submits their credentials, the application sends a POST request to the backend. If the request is successful, an authentication token is stored in `localStorage`. This token is then automatically included in future requests through the Authorization header, allowing access to protected sources. The frontend also performs basic client-side validation, such as checking required fields, verifying email format, and confirming password matching. On the backend, authentication behavior is managed through configuration files like `securityConfig.mjs`, which define request headers, API key requirements, and validation rules.

6.3.2. API Communication and Data Handling

All API communication is handled through a centralized API client (`apiClient.js`). This approach avoids repeating request logic across components and ensures consistency.

The API client is responsible for:

- Attaching headers (including authentication tokens)
- Parsing responses safely
- Handling errors in a structured way

It supports multiple request types (GET, POST, PUT, DELETE) and ensures that both JSON and non-JSON responses are processed without breaking the application. When an error occurs, a structured error object is returned, making it easier to debug issues and provide clear feedback to the user.

6.3.3. Session Management System

The application includes a session system that allows administrators to create and manage simulation sessions, while trainees can join and participate.

In the Admin Dashboard, the session lifecycle is controlled using React state, with three main stages: waiting, active, and ended. The interface updates dynamically based on the current state, displaying trainee participation, their stage, and final results.

The Create Session screen retrieves available scenarios from the backend and generates a session key. This key is used by trainees in the Join Session screen to access the session. While some elements currently use mock data, the overall structure is designed for full backend integration.

6.3.4. Scenario Decision System

The trainee interface is built around a decision-based system driven by structured scenario data. Each scenario consists of JSON nodes, and the application tracks the current node using state.

When a user selects an option, the application evaluates the outcome and either moves to another node or ends the scenario if a failure condition is reached. This data-driven approach separates logic from content, making the system easier to extend with new scenarios.

6.3.5. Reusable Components and Layout Architecture

The frontend is designed using reusable components and layout structures to maintain consistency across the application. For example, the Button component is used throughout the app and supports both navigation and custom actions.

Layout components such as FormLayout, HomeLayout, and AdminDashboardLayout provide consistent page structures while allowing individual screens to focus on their specific functionality. This separation improves readability, organization, and scalability.

6.3.6. Backend Logging and Security

The backend includes a structured logging and security setup. The audit logger (auditLogger.mjs) records important events while sanitizing input data to prevent unsafe values, such as prototype-related keys.

A centralized logger provides multiple log levels (info, warn, error, debug) with timestamps for better traceability. Configuration files like environmentConfig.mjs and securityConfig.mjs manage settings such as ports, headers, and API key requirements, making the system flexible across different environments.

6.3.7. Third-Party Libraries

The project uses several libraries to support core functionality:

- react-router-dom for client-side routing
- @mui/material for responsive layout design
- react-responsive-carousel for the homepage carousel

The backend relies mostly on custom-built modules but is structured in a way that allows additional libraries to be integrated if needed.

6.4. Database Implementation

This project utilizes a hybrid data storage approach by combining a relational SQL database with JSON (JavaScript Object Notation). This design was selected to balance the need for secure data management with the flexibility required for dynamic content.

The SQL database is used for managing user authentication and account data, while JSON is used to store scenario-based simulation data. This separation ensures that each type of data is handled using the most appropriate technology, improving both system efficiency and maintainability.

6.4.1. SQL Database for User Management

A relational SQL database is implemented to manage user accounts and authentication. This includes both Administrators and Trainees, as role selection occurs after login rather than being predefined within the database.

The database consists of a primary user table that stores the following attributes:

- Username
- Email address (unique)
- Password (hashed value)

This structure ensures that user data is stored securely and consistently. The use of a unique email constraint prevents duplicate accounts, while password hashing enhances security.

The SQL schema used for this implementation:

```
CREATE TABLE user (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL  
);
```

This schema provides a simple but effective structure for user management. The id field is the primary key and uniquely identifies each user, while the email field enforces uniqueness to prevent duplicate registrations. The password_hash field ensures that plaintext passwords are never stored in the database.

Although simple, the design provides a foundation for future expansion.

Figure 1: SIRE Database Creation Screenshot

Server: 127.0.0.1 > Database: sire

Table name: Add column(s)

Name	Type	Length/Values	Default	Collation	Attributes	Null	Index	Comments
id	INT		None			☐	PRIMARY	Primary Key
username	VARCHAR	50	None			☐		
email	VARCHAR	100	None			☐	UNIQUE	[email]
password_hash	VARCHAR	255	None			☐		

Table comments:
 Collation:
 Storage Engine:

PARTITION definition:

Partition by: (Expression or column list)

Partitions:

Console

Figure 2: SIRE Database "user" Table Schema Screenshot

Server: 127.0.0.1 > Database: sire > Table: user

Table structure

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None	Primary Key	AUTO_INCREMENT	Change Drop More
2	username	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
3	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
4	password_hash	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More

☐ Check all With selected: [Browse](#) [Change](#) [Drop](#) [Primary](#) [Unique](#) [Index](#) [Spatial](#) [Fulltext](#)

[Print](#) [Propose table structure](#) [Move columns](#) [Normalize](#)

Add column(s) after password_hash

Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	0	A	No	
Edit Rename Drop	email	BTREE	Yes	No	email	0	A	No	

Create an index on column(s)

Partitions

Console

6.4.2. JSON for Scenario and Application Data

Scenario data and simulation logic are stored using JSON files. These files define the structure of each scenario, including decision nodes, possible actions, outcomes, and progression paths.

Class A Fire – JSON Example

```
{
  "schema": "1.0",
  "title": "CLASS A FIRE — FULL BRANCHING ADVENTURE TREE",
  "category": "Class A Fire",
  "root": "A1",
  "phases": [
    { "id": 1, "name": "Refined Core Tree" },
    { "id": 2, "name": "Deep Expansion" },
    { "id": 3, "name": "Human + Edge-Case Exhaustion" }
  ],
  "nodes": {
    "A1": {
      "phase": 1,
      "title": "NODE A1 — Initial Observation",
      "situation": "A faint smell of smoke is reported on the second floor. No alarm is sounding. People nearby are unsure if it's serious.",
      "question": "What do you do first?",
      "options": [
        {
          "label": "A",
          "text": "Ignore it until flames are visible",
          "outcome": { "type": "node", "target": "A1A", "note": "Delay" }
        }
      ],
    },
  },
}
```

```
{
  "label": "B",
  "text": "Investigate the source of the smoke",
  "outcome": { "type": "node", "target": "A2", "note": "Correct investigation path" }
},
{
  "label": "C",
  "text": "Pull the fire alarm immediately",
  "outcome": { "type": "node", "target": "A1C", "note": "Early alarm path" }
},
{
  "label": "D",
  "text": "Call emergency services without checking",
  "outcome": { "type": "node", "target": "A1D", "note": "Premature emergency call path" }
}
]
},
"A1A": {
  "phase": 1,
  "title": "NODE A1A — Delay / Inaction",
  "situation": "Two minutes pass. The smell of smoke is stronger. Someone nearby jokes that it's
\"probably just someone burning popcorn.\".",
  "question": "What do you do now?",
  "options": [
    { "label": "A", "text": "Keep waiting to avoid embarrassment", "outcome": { "type": "node",
"target": "A1A1", "note": "Critical delay" } },
    { "label": "B", "text": "Investigate the source immediately", "outcome": { "type": "node", "target":
"A2", "note": "Late recovery" } },
```

```

    { "label": "C", "text": "Pull the fire alarm", "outcome": { "type": "node", "target": "A1C", "note":
"Alarm, but late" } },

    { "label": "D", "text": "Ask someone else to check while you stay put", "outcome": { "type":
"node", "target": "A1A2", "note": "Delegation confusion" }

    }

]

}, // ...

}

}

```

6.5. User Interface (UI) Design

The user interface (UI) for this application is designed using a component-based architecture in React. The design follows a structured layout approach, ensuring that users can easily navigate between different parts of the system.

The interface supports two primary user roles (Administrators and Trainees) and provides tailored workflows for each, including session creation, session participation, and scenario interaction.

6.5.1. Key UI Components

Layout Components

The application uses multiple reusable layout components to maintain a consistent structure across different screens. These include:

- **HomeLayout:** A split-screen layout that introduces the application, provides navigation options, and displays a scenario carousel.
- **FormLayout:** Used for authentication screens such as login and signup, with a clean input-focused design.
- **CreateSessionLayout and JoinSessionLayout:** Provide structured interfaces for session management tasks.
- **AdminDashboardLayout:** Supports monitoring and controlling session flow for administrators.
- **TraineeInterfaceLayout:** Designed for real-time interaction with scenario content.

6.5.2. Reusable Components

Several reusable components are implemented to improve modularity and maintainability:

- **Button Component:** Supports both navigation and custom event handling.
- **Footer Component:** Provides consistent branding and layout closure across all screens.
- **Carousel Component:** Displays rotating scenario previews on the home screen.

This modular approach allows components to be reused across multiple screens, improving code modularity and scalability.

6.5.3. User Interaction and Navigation

Navigation within the application is handled using client-side routing. Each major screen is mapped to a route, allowing users to move between pages without full page reloads.

User interaction is designed to be straightforward:

- **Authentication Flow:** Users can sign up and log in using form-based inputs with validation and error feedback.
- **Role Selection:** Users choose between Administrator and Trainee roles, directing them to different workflows.
- **Session Management:** Administrators can create sessions and monitor trainee progress, while trainees can join sessions using a session key.
- **Scenario Interaction:** Trainees interact with scenario content by selecting options presented as clickable cards, which dynamically update the simulation state.

State management is handled using React hooks, allowing the interface to update dynamically in response to user actions.

6.5.4. Design Approach

The interface is designed with usability as a primary focus. Design decisions include:

- **Consistent Layouts:** All screens follow a similar split-screen or panel-based structure.
- **Clear Visual Hierarchy:** Important information, such as scenario content, is emphasized using spacing and card-based layouts.
- **Immediate Feedback:** Users receive feedback through loading states, error messages, success messages, and updated content when actions are performed.
- **Simple Navigation:** Logical workflows guide users through tasks without confusion.

6.5.5. Visual Design

The application uses a dark theme with white and red accent colors to highlight borders and important elements. Card-based layouts are used to group related information, particularly in the admin dashboard and the trainee interface.

Figure 3: Home Screen Screenshot

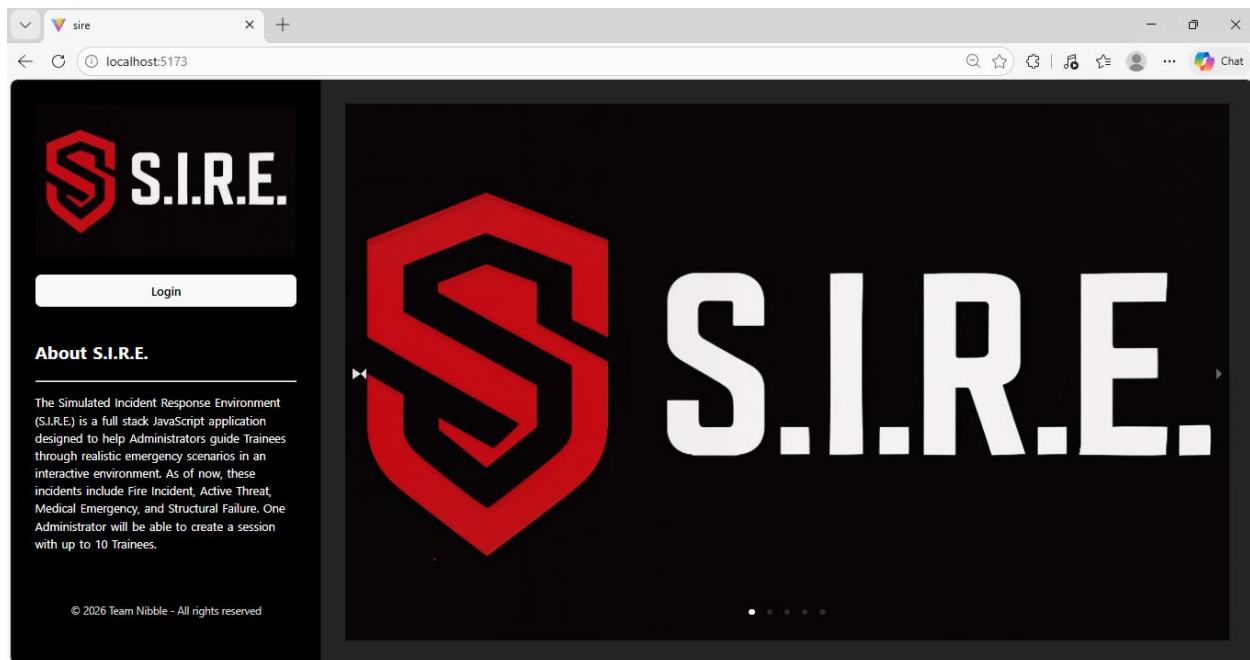


Figure 4: Create a Session Screen (With Validation) Screenshot

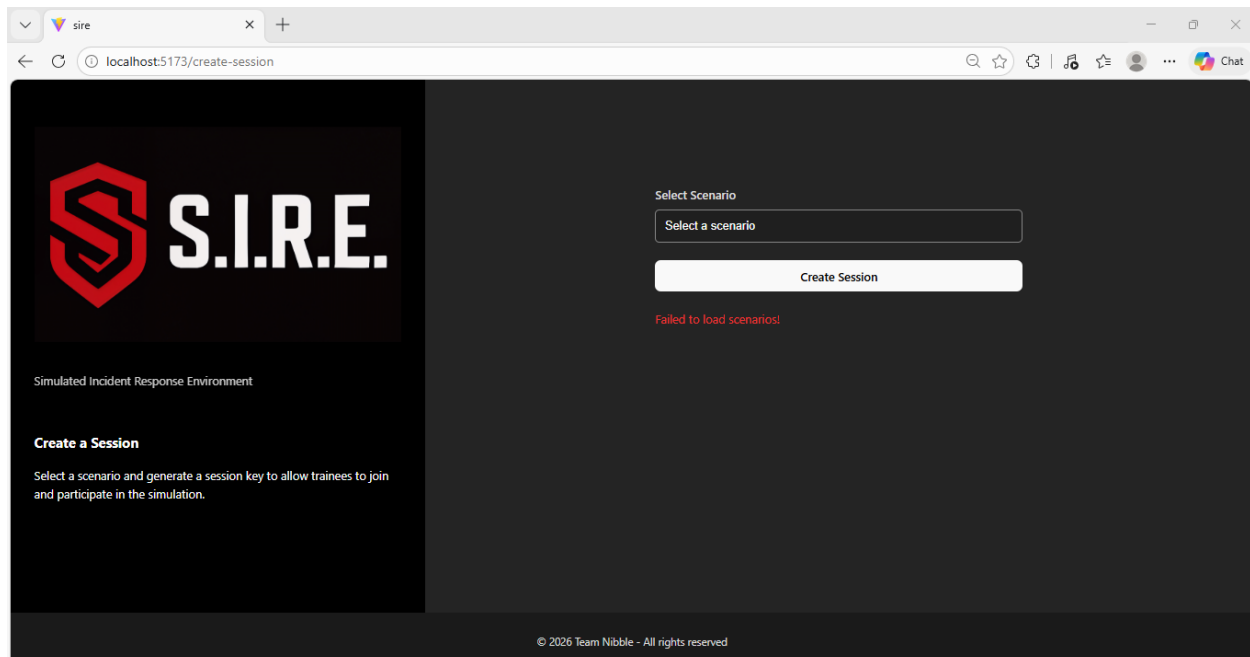


Figure 5: Administrator Dashboard (Before Session Start) Screenshot

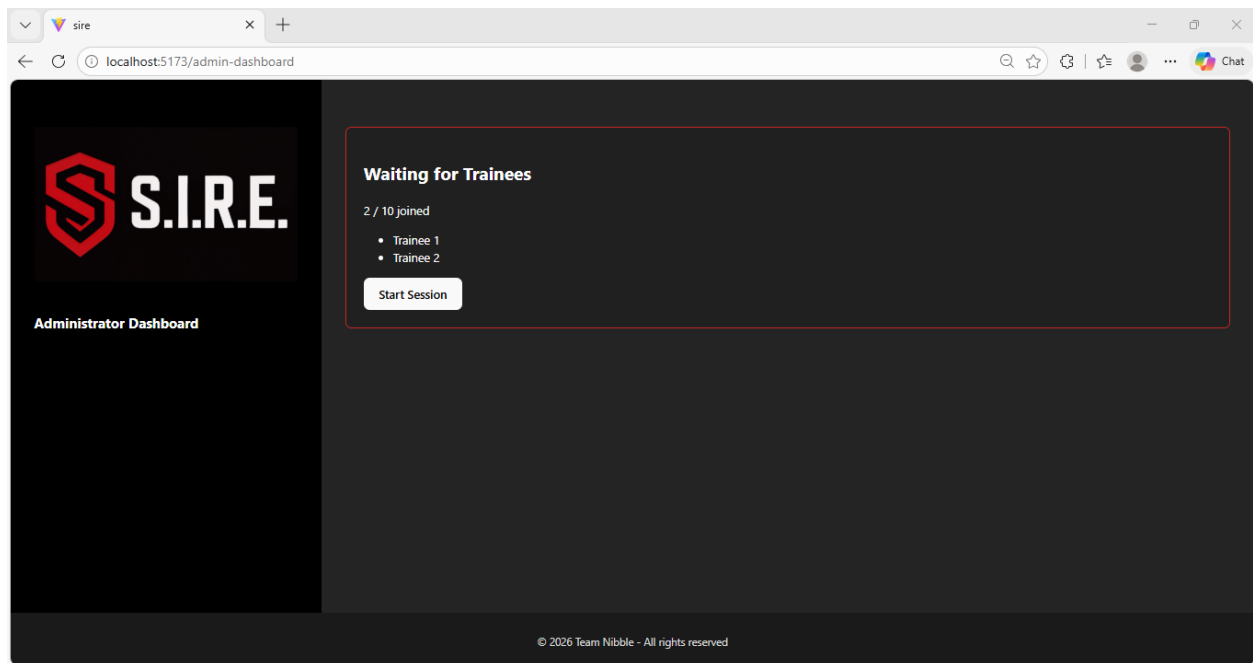
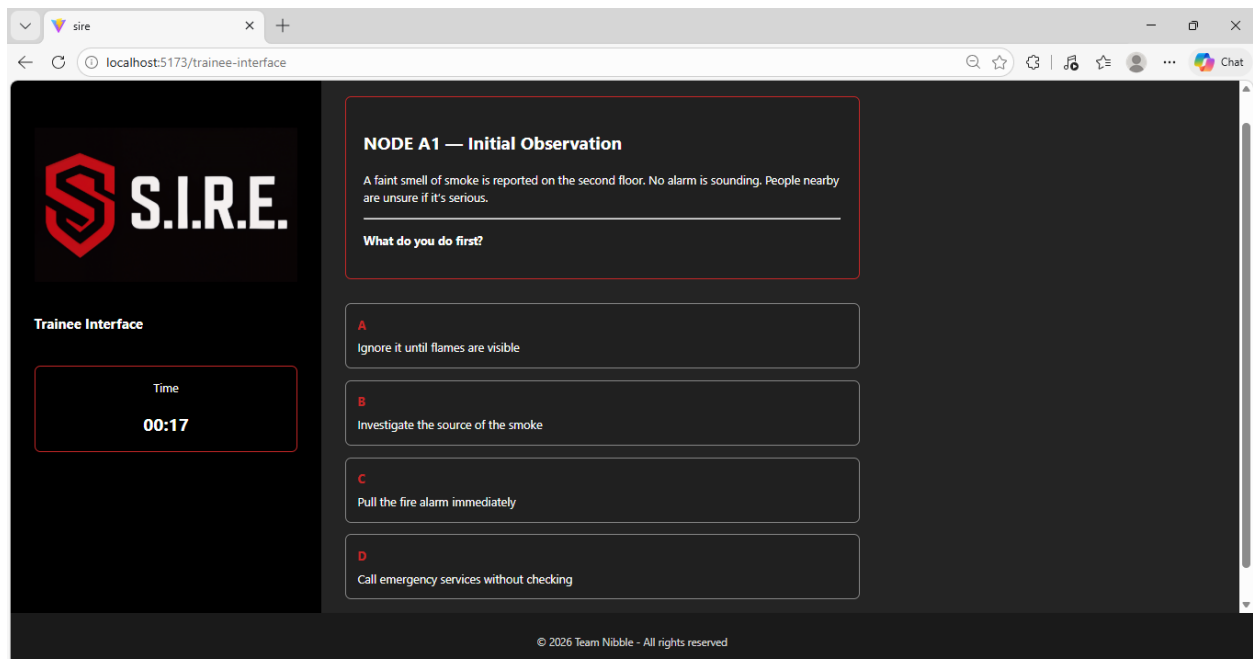


Figure 6: Trainee Interface (With Scenario) Screenshot



7. Testing

7.1. Testing Types

7.1.1. Frontend

The frontend was tested using a combination of manual and white-box testing techniques. Manual testing was used to verify UI layout responsiveness and navigation. White-box testing was applied during development to validate internal component logic, including form validation processes, control flow conditions, and state management within React components.

Automated testing has not yet been fully implemented, as integration between the frontend and backend is still in progress. Once the application is fully connected, automated testing tools such as Playwright may be introduced for end-to-end testing.

7.1.2. Backend

The backend was tested using a combination of integration testing and black-box testing techniques. Integration testing was used to verify that the individual server components, including REST API routes, authentication middleware, Socket.IO real-time communication, and session management, work correctly together as a complete system. Black-box testing was applied by sending HTTP requests and Socket.IO events to the running server and validating the responses, without relying on knowledge of the internal implementation.

Automated tests are written using Node.js's built-in assert module, with the Socket.io-client library used to simulate real-time client connections. Tests cover two areas: server integration tests (verifying REST API and Socket.IO behavior end-to-end) and CORS configuration tests (validating that cross-origin request handling is correctly enforced for both allowed and disallowed origins).

7.2. Test Cases and Results

7.2.1. Frontend

Resizing Test Cases

- Tested application across different screen sizes
- Adjusted height and width values to improve layout consistency
- Identified the need for additional refinement using minimum and maximum

Result: Layout is functional but requires minor improvements for full responsiveness

Navigation Test Cases

- Temporary navigation buttons were added to the home screen for testing
- Verified that all links correctly redirect to their intended pages
- Checked for broken or missing routes

Result: All navigation is functioning correctly, and no broken links or routing issues were found.

Frontend White-Box Test Case #1 – Internal Form Validation Logic

Test Case ID	FE-WB-01
Test Type	White-Box Testing
Component	src/screen/Login/index.jsx src/screen/Signup/index.jsx
Description	Verify that the internal validation logic is correctly implemented and checks user input before allowing form submission.
Preconditions	Form component is rendered and ready for input. Backend is not connected to the frontend.
Expected Result	Validation function prevents submission when inputs are invalid and allows submission only when inputs meet required conditions.
Actual Result	Form blocks invalid input and displays errors, while valid input successfully passes validation.
Status	Pass

7.2.2. Backend

Backend Integration Test Cases

Test Case ID	BE-IT-01
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/middleware/authMiddleware.mjs
Description	Verify that the REST API rejects requests without a valid API key and returns HTTP 401 Unauthorized.
Preconditions	Backend server is running. No API key is provided in the request headers.
Expected Result	Server returns HTTP 401 Unauthorized for requests without a valid x-api-key header.
Actual Result	Server correctly returned HTTP 401 for all requests missing or using an invalid API key.
Status	Pass

Test Case ID	BE-IT-02
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/services/sessionService.mjs
Description	Verify that an authenticated request to GET /api/session returns a list of active sessions, including a previously created session.
Preconditions	Backend server is running. A session has been created programmatically before the request. Valid API key is provided.
Expected Result	Server returns HTTP 200 with a JSON array of sessions. The pre-created session is present in the list.
Actual Result	Server returned HTTP 200 with a session array containing the expected session entry.
Status	Pass

Test Case ID	BE-IT-03
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/utls/validation.mjs
Description	Verify that passing an invalid query parameter (e.g. limit=not-a-number) to GET /api/session returns HTTP 400 Bad Request.
Preconditions	Backend server is running. Valid API key is provided. Query string contains an invalid limit value.
Expected Result	Server returns HTTP 400 Bad Request due to invalid query parameter.
Actual Result	Server correctly returned HTTP 400 when an invalid limit value was provided.
Status	Pass

Test Case ID	BE-IT-04
Test Type	Integration Testing / Black-Box Testing
Component	src/sockets/socketServer.mjs src/middleware/authMiddleware.mjs
Description	Verify that a Socket.IO client can authenticate using the legacy extraHeaders method (backward-compatible path) and successfully join a session.
Preconditions	Backend server is running with Socket.IO. A session exists. API key is passed in extraHeaders.
Expected Result	Client connects successfully and receives a session:joined event with the correct session code.
Actual Result	Client connected and received session:joined with the expected session code and timeline index of -1.
Status	Pass

Test Case ID	BE-IT-05
Test Type	Integration Testing / Black-Box Testing
Component	src/sockets/socketServer.mjs src/middleware/authMiddleware.mjs
Description	Verify that a Socket.IO client can authenticate using the auth payload method (browser-safe primary path) and successfully join a session.
Preconditions	Backend server is running with Socket.IO. A session exists. API key is passed in the auth object.
Expected Result	Client connects successfully and receives a session:joined event with the correct session code.
Actual Result	Client connected and received session:joined with the expected session code.
Status	Pass

Test Case ID	BE-IT-06
Test Type	Integration Testing / Black-Box Testing
Component	src/sockets/socketServer.mjs src/middleware/authMiddleware.mjs
Description	Verify that a Socket.IO connection attempt using a wrong API key is rejected with an UNAUTHORIZED error.
Preconditions	Backend server is running with Socket.IO. An incorrect API key is used in the auth payload.
Expected Result	Connection is refused and the client receives a connect_error event containing the message UNAUTHORIZED.
Actual Result	Connection was refused and the connect_error event contained the UNAUTHORIZED message as expected.
Status	Pass

Test Case ID	BE-IT-07
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/middleware/authMiddleware.mjs
Description	Verify that DELETE /api/session/:code requires both an API key and a ticket ID header, and that omitting the ticket ID returns HTTP 400.
Preconditions	Backend server is running. A session exists. API key is provided but ticket ID header is omitted.
Expected Result	Server returns HTTP 400 when the ticket ID header is missing.
Actual Result	Server correctly returned HTTP 400 when the x-ticket-id header was absent.
Status	Pass

Test Case ID	BE-IT-08
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/services/sessionService.mjs
Description	Verify that DELETE /api/session/:code with both a valid API key and ticket ID successfully deletes the session and removes it from subsequent session list responses.
Preconditions	Backend server is running. A session exists. Both API key and ticket ID headers are provided.
Expected Result	Server returns HTTP 200 and the session no longer appears in a subsequent GET /api/session response.
Actual Result	Server returned HTTP 200 and the session was absent from the subsequent session list.
Status	Pass

Test Case ID	BE-CT-01
Test Type	Integration Testing / Configuration Testing
Component	src/config/environmentConfig.mjs
Description	Verify that the environmentConfig module correctly parses the ALLOWED_ORIGINS environment variable and exposes it as an array.
Preconditions	ALLOWED_ORIGINS environment variable is set. The environmentConfig module is imported.
Expected Result	The allowedOrigins property is an array.
Actual Result	allowedOrigins was confirmed to be an array.
Status	Pass

Test Case ID	BE-CT-02
Test Type	Integration Testing / Configuration Testing
Component	src/server.mjs (CORS middleware configuration)
Description	Verify that the CORS middleware returns correct headers for a request from an allowed origin, including credentials support.
Preconditions	Express app is configured with a restricted list of allowed origins. Request is sent from an allowed origin.
Expected Result	Response includes Access-Control-Allow-Origin matching the request origin and Access-Control-Allow-Credentials set to true.
Actual Result	Response correctly included Access-Control-Allow-Origin set to the allowed origin and credentials header set to true.
Status	Pass

Test Case ID	BE-CT-03
Test Type	Integration Testing / Configuration Testing
Component	src/server.mjs (CORS middleware configuration)
Description	Verify that the CORS middleware does not return CORS headers for requests from disallowed origins.
Preconditions	Express app is configured with a restricted list of allowed origins. Request is sent from a disallowed origin.
Expected Result	Response does not include an Access-Control-Allow-Origin header.
Actual Result	Response contained no Access-Control-Allow-Origin header for the disallowed origin.
Status	Pass

Test Case ID	BE-CT-04
Test Type	Integration Testing / Configuration Testing
Component	src/server.mjs (CORS middleware configuration)
Description	Verify that wildcard CORS configuration allows requests from any origin without setting credentials headers.
Preconditions	Express app is configured with wildcard CORS (origin: "*"). Request is sent from any arbitrary origin.
Expected Result	Response includes Access-Control-Allow-Origin set to * and does not include Access-Control-Allow-Credentials.
Actual Result	Response correctly returned Access-Control-Allow-Origin: * without a credentials header.
Status	Pass

8. Conclusion

8.1. Overview

The SIRE (Simulated Incident Response Environment) project was a challenging yet rewarding experience that brought together technical development, planning, and teamwork. As a group, we successfully delivered a functional prototype that demonstrates the core concept of an interactive emergency response training system, including a React-based frontend, a Node.js and Express backend, real-time communication using Socket.IO, and scenario-driven logic built through structured JSON scripts.

8.2. What Went Well

One of the team's greatest strengths was our ability to define clear roles and ensure that each team member contributed within their area of responsibility. The frontend, backend, and scenario systems were all developed with modern technologies and aligned toward a shared goal. Additionally, the project was supported by tools such as GitHub and Jira, which improved the documentation and project workflows. Despite balancing multiple courses, we were able to produce a working system and present it at the Tech Showcase at the NSCC Ivany Campus. The collaboration between technical and non-technical contributions was also a key factor in the project's success.

8.3. What Went Wrong

While the project was successful overall, there were several challenges. Communication between team members was inconsistent and often increased only near deadlines, which made coordination and integration more difficult. Much of the work was completed independently and merged later, leading to delays in connecting the frontend, backend, and scenario components. Additionally, real-time collaboration or meetings were less frequent than expected. On the technical side, some features (such as real-time inject functionality) were not fully refined, and testing remained manual with little to no implementation of automated tools or CI/CD pipelines.

8.4. What We Would Do Differently

If we were to approach this project again, we would prioritize stronger and more consistent communication from the beginning, including regular check-ins and clearer progress updates. Earlier integration between all system components would be a major focus to reduce last-minute issues. We would also establish stricter standards for using project management tools and enforce consistent workflows across the team. From a technical perspective, we would aim to implement automated testing, improve validation and scenario logic, and enhance UI accessibility and responsiveness.

9. References

1. GeeksforGeeks. (2018, July 12). *White box Testing Software Engineering*. GeeksforGeeks. <https://www.geeksforgeeks.org/software-testing/software-engineering-white-box-testing/>
2. GeeksforGeeks. (2023, December 27). *React Tutorial*. GeeksforGeeks. <https://www.geeksforgeeks.org/reactjs/react/>
3. GeeksforGeeks. (2024, January 24). *Node.js Tutorial*. GeeksforGeeks. <https://www.geeksforgeeks.org/node-js/nodejs/>
4. GitHub Contributors. (n.d.). *Conventional Commits*. Conventional Commits. <https://www.conventionalcommits.org/en/v1.0.0/>
5. Google. (2024). *Chatbot AI - Powered by Multi-Model AI Technologies & AI Assistant*. Chatbotai.co. <https://chatbotai.co/>
6. Microsoft. (2026). *Software Development Life Cycle | Microsoft Power Automate*. Microsoft.com. <https://www.microsoft.com/en-us/power-platform/topics/phases-of-the-software-development-lifecycle?msockid=1679ddde67bd62d92639cafb66d2631f>
7. Microsoft Copilot. (2025). *Microsoft Copilot*. Microsoft Copilot; Microsoft. <https://copilot.microsoft.com/>
8. SimplyJavaScript. (2026). *JavaScript Coding Guidelines & Style Guide (2026 Best Practices)*. Simplyjavascript.com; SimplyJavaScript. <https://simplyjavascript.com/guidelines.html>
9. Superhuman. (2026). *Grammarly*. Grammarly.com; Grammarly. <https://www.grammarly.com/?msockid=1679ddde67bd62d92639cafb66d2631f>
10. W3Schools. (2019). *JavaScript Style Guide*. W3schools.com. https://www.w3schools.com/js/js_conventions.asp
11. Draw.io. (2024). *Diagram Software and Flowchart Maker*. Www.drawio.com. <https://www.drawio.com/>
12. GeeksforGeeks. (2017, October 26). *Activity Diagrams Unified Modeling Language (UML)*. GeeksforGeeks. <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-activity-diagrams/>
13. GeeksforGeeks. (2023, November 24). *Use Case Diagram Unified Modeling Language (UML)*. GeeksforGeeks. <https://www.geeksforgeeks.org/system-design/use-case-diagram/>
14. GeeksforGeeks. (2023, December 29). *What is a Mockup & Why do we need one ?* GeeksforGeeks. <https://www.geeksforgeeks.org/techtips/what-is-a-mockup-why-do-we-need-one/>
15. GeeksforGeeks. (2024, January 24). *What is Wireframing: A Complete Guide*. GeeksforGeeks. <https://www.geeksforgeeks.org/blogs/what-is-wireframing/>
16. Grammarly. (2025). *Grammarly*. Grammarly.com. <https://app.grammarly.com/>
17. Talreja, A. (2024, July 9). *Software Development Lifecycle (SDLC) - Design Phase – Nextra*. Teachingagile.com. <https://teachingagile.com/sdlc/design>

18. Visual Paradigm. (n.d.). *What is Entity Relationship Diagram (ERD)?* Visual-Paradigm.com. <https://www.visual-paradigm.com/guide/data-modeling/what-is-entity-relationship-diagram/>
19. Visual Paradigm. (2019). *What is Component Diagram?* Visual-Paradigm.com. <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/>
20. Asana. (2022, February 18). *Team Charter Template: A Roadmap for Team Success* • Asana. Asana. <https://asana.com/resources/team-charter-template>
21. Asana. (2022, May 6). *How to Write a Software Requirement Document (With Template)* • Asana. Asana. <https://asana.com/resources/software-requirement-document-template>
22. Atlassian. (2024). *Jira*. Atlassian. <https://www.atlassian.com/software/jira>
23. Draw.io. (2024). *Diagram Software and Flowchart Maker*. Www.drawio.com. <https://www.drawio.com/>
24. Drumond, C. (2024). *What is scrum and how to get started*. Atlassian. <https://www.atlassian.com/agile/scrum>
25. GeeksforGeeks. (2024, January 2). *What is Gantt Chart in Project Management?* GeeksforGeeks. <https://www.geeksforgeeks.org/business-studies/what-is-gantt-chart-in-project-management/>
26. GitHub Contributors. (2025). *GitHub*. GitHub. <https://github.com/>
27. Grammarly. (2025). *Grammarly*. Grammarly.com. <https://app.grammarly.com/>
28. Mermaid Contributors. (2025). *Mermaid | Diagramming and charting tool*. Mermaid.js.org. <https://mermaid.js.org/>
29. Microsoft. (2024). *Response to the prompt "How to refine the documentation structure for a project interim report?"* [Large language model AI assistant]. <https://copilot.microsoft.com>
30. Raeburn, A. (2025, February 7). *Work Breakdown Structure (WBS): What Is It and How Do You Use It?* • Asana. Asana. <https://asana.com/resources/work-breakdown-structure>

Appendix A – Gantt Chart

Table Overview

WBS	Activity	Duration	Start	End	Depends On	Owner(s)	Milestone
1.0	Team Formation	7 days	Jan 5	Jan 11	-	Kael Payette	Yes
1.1	Assign roles and responsibilities	2 days	Jan 5	Jan 6	1.0	Kael Payette	No
1.2	Establish communication channels	2 days	Jan 7	Jan 8	1.1	Kael Payette	No
1.3	Set up workflow and collaboration standards	3 days	Jan 9	Jan 11	1.2	Kael Payette	No
2.0	Project Proposal & GitHub Repository	7 days	Jan 12	Jan 18	1.0	Kael	Yes
2.1	Draft project proposal	3 days	Jan 12	Jan 14	2.0	Kael	No
2.2	Create GitHub repository	1 day	Jan 15	Jan 15	2.1	Kael	No
2.3	Set up branching and versioning standards	3 days	Jan 16	Jan 18	2.2	Kael	No
3.0	Team Charter, Project Charter & Initial Documents	7 days	Jan 19	Jan 25	2.0	All Members	Yes
3.1	Team Charter drafting	3 days	Jan 19	Jan 21	3.0	Kael	No
3.2	Project Charter drafting	3 days	Jan 22	Jan 24	3.1	Kael, Leon	No
3.3	Review and revisions	1 day	Jan 25	Jan 25	3.2	All Members	No
4.0	Kick-Off Meeting, Analysis, WBS, Activity Diagram & Gantt Chart	7 days	Jan 26	Feb 1	3.0	Kael, Leon	Yes
4.1	Kick-off meeting	1 day	Jan 26	Jan 26	4.0	All Members	No
4.2	Requirements analysis	2 days	Jan 27	Jan 28	4.1	All Members	No
4.3	WBS creation	2 days	Jan 29	Jan 30	4.2	Kael	No

4.4	Activity diagram & Gantt chart	2 days	Jan 31	Feb 1	4.3	Leon	No
5.0	DB Design, UI Design, Scenario Research & Scripting	7 days	Feb 2	Feb 8	4.0	Leon, John	No
5.1	Database schema design (Node.js + free-tier hosting)	2 days	Feb 2	Feb 3	5.0	Leon	No
5.2	UI wireframes (React layouts)	2 days	Feb 4	Feb 5	5.1	Leon	No
5.3	Scenario research	2 days	Feb 6	Feb 7	5.2	John	No
5.4	Scenario script drafting	1 day	Feb 8	Feb 8	5.3	John	No
6.0	Backend & Frontend Implementation – Scenario 1	7 days	Feb 9	Feb 15	5.0	Tom, Leon	No
6.1	Backend setup (Node.js, Express)	2 days	Feb 9	Feb 10	6.0	Tom	No
6.2	WebSocket engine (Socket.IO)	2 days	Feb 11	Feb 12	6.1	Tom	No
6.3	Frontend components (React)	2 days	Feb 13	Feb 14	6.2	Leon	No
6.4	Session-based access (Admin/Trainee)	1 day	Feb 15	Feb 15	6.3	Tom, Leon	No
7.0	Testing & Changes – Scenario 1	7 days	Feb 16	Feb 22	6.0	John	No
7.1	Script validation	2 days	Feb 16	Feb 17	7.0	John	No
7.2	Functional testing	3 days	Feb 18	Feb 20	7.1	John	No
7.3	Fixes and refinements	2 days	Feb 21	Feb 22	7.2	Tom, Leon	No
8.0	Backend & Frontend Implementation – Scenario 2	7 days	Feb 23	Mar 1	7.0	Tom, Leon	No
8.1	Backend logic	3 days	Feb 23	Feb 25	8.0	Tom	No
8.2	Frontend UI	3 days	Feb 26	Feb 28	8.1	Leon	No

8.3	Real-time sync integration	1 day	Mar 1	Mar 1	8.2	Tom, Leon	No
9.0	Testing & Changes – Scenario 2	7 days	Mar 2	Mar 8	8.0	John	No
10.0	Backend & Frontend Implementation – Scenario 3	7 days	Mar 9	Mar 15	9.0	Tom, Leon	No
11.0	Testing & Changes – Scenario 3	7 days	Mar 16	Mar 22	10.0	John	No
12.0	Backend & Frontend Implementation – Scenario 4	7 days	Mar 23	Mar 29	11.0	Tom, Leon	No
13.0	Testing & Changes – Scenario 4	7 days	Mar 30	Apr 5	12.0	John	No
14.0	Final Testing & Project Summarization	7 days	Apr 6	Apr 12	13.0	Kael, John	Yes
14.1	Full system QA	3 days	Apr 6	Apr 8	14.0	John	No
14.2	Documentation package	3 days	Apr 9	Apr 11	14.1	Kael	No
14.3	Deployment verification (free-tier hosting)	1 day	Apr 12	Apr 12	14.2	Tom, Leon	No
15.0	Final Presentation & Project Closure	3 days	Apr 13	Apr 15	14.0	Kael, Leon	Yes
15.1	Presentation preparation	2 days	Apr 13	Apr 14	15.0	Kael, Leon	No
15.2	Final demo	1 day	Apr 15	Apr 15	15.1	All Members	No

Diagram Overview

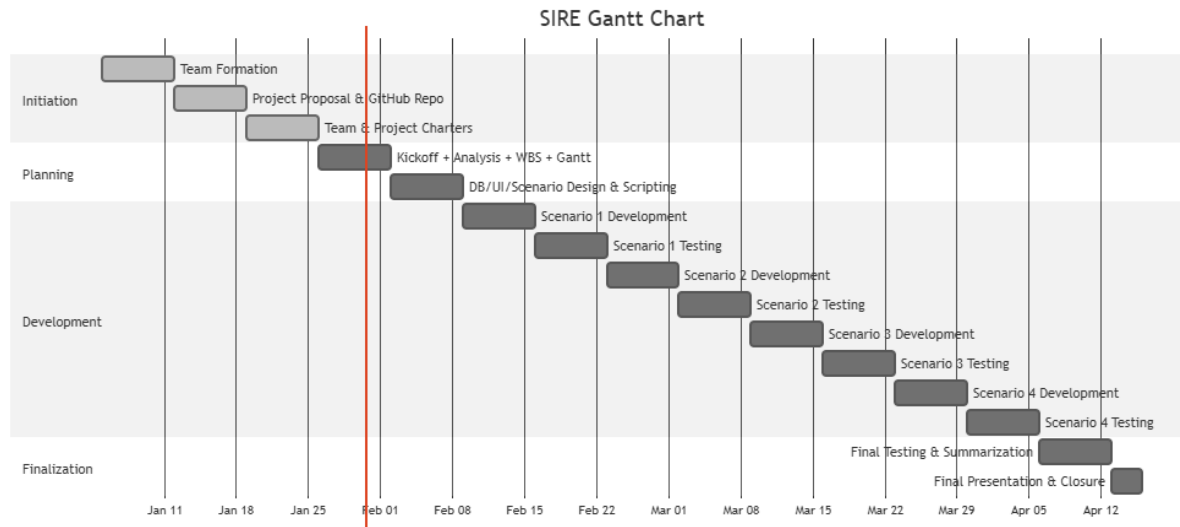
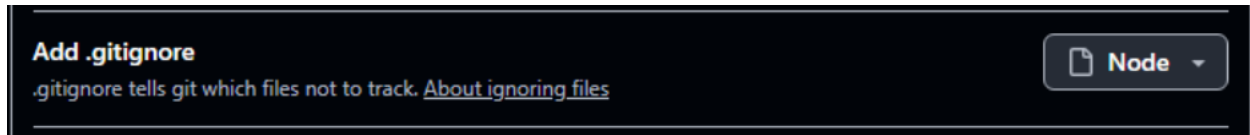


Figure 7: Created with the help of Draw.io and Mermaid

Appendix B – GitHub Repository

Repository Structure

A GitHub repository titled “sire-simulator” was created and set up with a .gitignore file.



The GitHub page will be used to keep track of all project components:

- **backend** – Supports backend development and versioning.
- **frontend** – Supports frontend development and versioning.
- **docs** – Stores all non-code project resources for the project.
- **scenarios** – Provides a dedicated space for scenario design and planning.
- **test** – Facilitates test case development and automated testing workflows.

The GitHub repo is split into 2 branches:

- **main** – The stable branch containing finalized/working code. Only approved commits and merges are permitted.
- **dev** – The integration branch where all feature branches merge first. Serves as the active development branch.

Created per task by using this format: Feature/<developer>/<feature-name>

Examples:

- feature/tom/socket-engine
- feature/leon/admin-dashboard
- feature/john/scenario-fire
- feature/kael/documentation

Commit Standards

Good commit:

✓ “Added timed escalation scheduler for scenarios.”

Bad commit:

✗ “added more”

Appendix C – Project Minutes

Weeks 1-4

Dates	Hours	Times	Description	Present
5-Jan-26	2	10:30 - 11:30 12:30 - 13:30	Forming group and discussing Capstone ideas	J, K, L, T
7-Jan-26	2	10:30 - 11:30 12:30 - 13:30	Coming up and agreeing on a Capstone project	J, K, L, T
12-Jan-26	2	10:30 - 11:30 12:30 - 13:30	Going over project proposal document draft	J, K, L, T
14-Jan-26	2	10:30 - 11:30 12:30 - 13:30	Agreeing on Proposal 2 (JavaScript)	J, K, L, T
21-Jan-26	2	10:30 - 11:30 12:30 - 13:30	Going over project documents (charters)	J, K, L, T
28-Jan-26	2	10:30 - 11:30 12:30 - 13:30	Reviewing project documents and talking with stakeholder	K, L, T

Hours: 12 hours

Weeks 5-8

Dates	Hours	Times	Description	Present
2-Feb-26	2	10:30 - 11:30 12:30 - 13:30	Working on the Kick-off meeting slides and reviewing progress	J, K, L, T
2-Apr-26	2	10:30 - 11:30 12:30 - 13:30	Holding Kick-off meeting and working on individual tasks	J, K, L, T
9-Feb-26	2	10:30 - 11:30 12:30 - 13:30	Working on individual parts of the project according to the workplan	J, K, L, T
11-Feb-26			NO SCHOOL (Snowday)	
16-Feb-26			NO SCHOOL (Holiday)	
18-Feb-26	2	10:30 - 11:30 12:30 - 13:30	Working on individual parts of the project according to the workplan	K, L, T
23-Feb-26			NO CLASS	
25-Feb-26			NO CLASS	

Hours: 8 hours

Weeks 9-13

Dates	Hours	Times	Description	Present
2-Mar-26	2	10:30 - 11:30 12:30 - 13:30	Gathering assets for the Home screen, including images for the image carousel	J, K, L, T
4-Mar-26	2	10:30 - 11:30 12:30 - 13:30	Creating the SIRE logo for project use and mixing background music for active sessions	J, K, L, T
9-Mar-26	2	10:30 - 11:30 12:30 - 13:30	Creating a mockup Tech Showcase poster with a general layout in Canva	J, K, L, T
11-Mar-26	2	10:30 - 11:30 12:30 - 13:30	Improving the Tech Showcase poster and developing additional frontend components	J, K, L, T
19-Mar-26	3	13:00 - 16:00	Developing the Tech Showcase poster layout and written content	J, K, L, T
23-Mar-26	2	10:30 - 11:30 12:30 - 13:30	Integrating visuals of the SIRE app into the Tech Showcase poster	J, K, L, T
25-Mar-26	2	10:30 - 11:30 12:30 - 13:30	Preparing for the Tech Showcase by Completing the Poster and the Documentation Folder	J, K, L, T
26-Mar-26	6	13:00 - 19:00	Attending the Tech Showcase at NSCC's Ivany Campus in Dartmouth	J, K, L, T
30-Mar-26	2	10:30 - 11:30 12:30 - 13:30	Preparing for the Prototype Demonstration by Completing the Workflow and Presentation Slides	J, K, L, T
1-Apr-26	2	10:30 - 11:30 12:30 - 13:30	Conducting the Prototype Demonstration using the Presentation Slides and the SIRE Prototype	J, K, L

Hours: 25 hours

Total Hours: 45 hours

Appendix D – Project Meetings

Meeting #1

Project Details

Project Title	S.I.R.E. (Simulated Incident Response Environment)
Meeting Date	2026-01-07
Time	10:30:00 – 11:30:00 & 12:30:00 – 13:30:00
Location	COGS – Room 115
Attendees	John H., Kael P., Leon W., Thomas B.
Absentees	None

Agenda Items

Agenda Item	Discussion Points	Decisions Made	Action Items
#1: Deciding on a Capstone Project	- AI/ML & Responsible GenAI Applications - Cybersecurity & Digital Trust - Health Informatics & TEFCA-Ready Interoperability - FinTech & Open Banking - Climate & Coastal Resilience - Energy & Mobility - EdTech, Skills & Labor Market - Web-Based Response Simulator	Selected Web-Based Response Simulator	- Define project need and purpose - Create project proposal - Conduct market research and analysis - Identify stakeholders and target users - Define project scope and breakdown - Identify deliverables, goals, and required resources
#2: Deciding on the Project Name	- I.R.S. (Incident Response Simulator) - S.I.R.E. (Simulated Incident Response Environment)	Selected S.I.R.E., as "I.R.S." may be confused with the Internal Revenue Service.	Ensure consistent use of the name across all documents and artifacts
#3: Deciding on the roles	- Selection of Project Manager (via vote) - Division of work into four core roles	John Hay – Script Master / QA	Develop Work Breakdown Structure (WBS)

	• Assignment based on strengths	Scenario writing, integration testing, accessibility review • Kael Payette – Project Manager Coordination, documentation, Git & Jira workflows, project standards • Leon Wasiliew – Frontend & UI/UX Lead React UI, layouts, real-time state, responsive design • Tom Burchell – Backend & Real-Time Lead Socket.IO, scenario engine, server architecture, CI/CD, cloud deployment	• Create project schedule and Gantt chart • Build foundational knowledge in assigned areas • Maintain consistent effort on action items
#4: Deciding on the Team Name	• Backend Boys • Quarter Byte • Nibble	• Selected Nibble, as “Backend Boys” sounds inappropriate and “Quarter Byte” is not very memorable.	• Use the team name consistently across all documents and artifacts

Project Status Updates

Progress	The project initiation phase is almost done. We just need to finish the Project Proposal and the Project Charter.
Challenges	Since we do not have a lot of experience in this area, it's easy to misjudge the project scope. Also, not keeping good documentation could lead to confusion or things getting off track.
Next Steps	We will work on the Project Proposal first, then the Project Charter and Team Charter. We will also start preparing for the Kick-off Meeting.

Risks/Issues Identified

Risk/Issue	Mitigation Plan
Communication gaps, especially in early decision-making stages	- Use MS Teams channel “The Floor Horseman” actively - Encourage regular updates from all members - Ensure all members contribute before major decisions
Poor initial planning may cause rework, delays, and scope issues	- Develop an early WBS draft - Create a preliminary Action Item list - Review for gaps or missing components

Next Meeting

Date of Next Meeting	Details
2026-02-04	It will be the Kick-off meeting, where we will prepare a PowerPoint presentation and present our initial plan for the Planning Phase.

Meeting #2

Project Details

Project Title	S.I.R.E. (Simulated Incident Response Environment)
Meeting Date	2026-02-04
Time	10:30:00 – 11:30:00
Location	COGS – Room 115
Attendees	John H., Kael P., Leon W., Thomas B.
Absentees	None

Agenda Items

Agenda Item	Discussion Points	Decisions Made	Action Items
#1: Welcome & Introductions	Introduced team members and reviewed roles	Confirmed roles with the Project Stakeholder	Ensure the Project Stakeholder understands each team member's responsibilities
#2: Project Context	Overview of the S.I.R.E. project and its purpose	Confirmed project direction and importance	Keep development aligned with project goals
#3: Objectives (SMART)	Defined clear and measurable project objectives	Agreed on SMART goals for the project	Follow and track objectives throughout development
#4: Incident Response Scenarios	- Discussed possible training scenarios - Focus on real-world applicability	Proceed with core scenarios that apply across industries	Begin developing and refining scenarios
#5: Architecture Discussion	- Discussed frontend and backend technologies - Reviewed system structure	Use React (Vite) for frontend and Node.js (Express) for backend	Continue system setup and architecture planning
#6: Risk Discussion	- Sync drift → early Socket.IO testing - Scope creep → stick to MVP - Dependency conflicts → use lockfiles - Tight timeline → work in parallel	- Approved mitigation strategies - Will continue tracking new risks	Apply mitigation strategies and maintain communication

#7: Timeline Discussion	Reviewed milestones and deadlines	Follow WBS and Gantt Chart	Track progress and update timeline as needed
#8: Next Steps Discussion	Outlined immediate next steps after kick-off	Focus on development, documentation, and coordination	- Continue frontend work and documentation - Prepare for next meeting

Project Status Updates

Progress	The kick-off presentation was completed during the first part of the meeting. The remaining time was spent planning the UI and beginning work with React. The team has also made solid progress between meetings in research, setup, and documentation.
Challenges	Some parts of the project planning are not fully aligned yet. There is also a risk of multiple versions of documents causing confusion.
Next Steps	The team will continue working on frontend development and UI design. Documentation will be updated and standardized in the GitHub repository. Preparation for the next progress meeting will also begin.

Risks/Issues Identified

Risk/Issue	Mitigation Plan
Real-Time Sync Drift	Begin early testing with Socket.IO to detect and correct synchronization issues before full implementation
Scope Creep	Define the scope and ensure all features and tasks stay within it
Dependency Conflicts	Use pinned package versions and lockfiles to prevent version mismatches
Tight Timeline	Assign tasks clearly and encourage parallel work where possible

Next Meeting

Date of Next Meeting	Details
2026-03-19	The progress meeting during March break will focus on preparing for the Pre-Tech Showcase, including confirming the poster design and required documentation.

Meeting #3

Project Details

Project Title	S.I.R.E. (Simulated Incident Response Environment)
Meeting Date	2026-03-19
Time	13:00:00 – 16:00:00
Location	COGS – Room 115
Attendees	John H., Kael P., Leon W., Thomas B.
Absentees	None

Agenda Items

Agenda Item	Discussion Points	Decisions Made	Action Items
#1: Poster and Documentation Preparation	- Decided on poster content, layout, and size - Reviewed what needs to be included in the project folder for the Tech Showcase	- Poster design finalized - Leon W. will manage the Documentation Folder and ensure all files are ready	- Complete poster printing - Prepare and organize files in the Documentation Folder - Ensure static frontend version is ready
#2: Frontend and UI Design	- Reviewed wireframes, mockups, and UI structure - Discussed React setup, folder structure, and components (Home, Signup, Login, etc.)	- Continue working on UI components for consistency and style - Static frontend version should be ready for demo	- Finish the minimum frontend components - Prepare static version for Tech Showcase
#3: Scenario Scripting and Showcase Demo	- Reviewed scenario scripts and background music - Checked backend workflows (session creation and joining) - Connected backend to the frontend for the static demo	Use completed scenarios and backend workflows in the Tech Showcase demo	- Test backend and frontend to have a complete demo - Finalize scenarios and poster content for showcase

Project Status Updates

Progress	The poster design has been completed, and the Documentation Folder has been assigned. The frontend static version is ready, and the backend has been successfully integrated with the frontend for the demo.
Challenges	Coordinating poster updates with the frontend and backend work has been challenging. Ensuring that all static content is ready and consistent for the showcase has required careful attention. Additionally, managing last-minute refinements has been a continuing challenge.
Next Steps	The team will print the poster and finalize the Documentation Folder. They will also test and polish the frontend and backend demo to ensure everything works smoothly. Finally, the team will prepare for the Tech Showcase.

Risks/Issues Identified

Risk/Issue	Mitigation Plan
Time Pressure for Tech Showcase	Prioritize critical tasks, work in parallel, and monitor progress closely to stay on schedule
Integration Bugs Between Frontend and Backend	Test the static frontend with the backend demo thoroughly and fix issues promptly
Poster or Documentation Errors	Review and finalize all poster and documentation materials ahead of the showcase to avoid mistakes

Next Meeting

Date of Next Meeting	Details
None	None